

Automation and AI in Software Development: Research Trends, Advantages, Disadvantages, Threats, and Opportunities

Antti Kiviniemi

Master's Thesis



UNIVERSITY
OF EASTERN
FINLAND

School of Computing

Computer Science

May 2024

UNIVERSITY OF EASTERN FINLAND, Faculty of Science, Forestry and Technology, Joensuu

School of Computing

Computer Science

Student, Antti Kiviniemi: Automation and AI in Software Development: Research Trends, Advantages, Disadvantages, Threats, and Opportunities

Master's Thesis, 60 pages

Thesis supervisor: Professor Matti Tedre

May 2024

Abstract

There is an emergence of artificial intelligence (AI) technologies, which can be utilized for automating software development processes in a generalizable and scalable way, making software development processes more efficient. The goal of this paper is to map out the state of research in the field, identifying themes and trends; and identify what software development life cycle (SDLC) processes AI and automation are applicable to, what are the advantages of this, and what problems and opportunities follow. Research methods included bibliometric analysis and selective literature synthesis and analysis. The conducted bibliometric analysis identified a growing trend in publications and a mixed trend in citations, the earlier research having an enduring impact. The most cited articles discussed topics of automation and AI in debugging, testing, and code writing. In the literature synthesis and analysis, AI-tools were found to have applications in all SDLC phases, increasing the efficiency of processes, with the biggest problems found in reliability, security, and implementation of the technology. Job displacement effects were not found to be a significant problem, while automation was found to offer a solution to a current workforce shortage in the Finnish software development sector. Problems connected to using AI were found to be solvable by practical means. AI is a rapidly growing tech theme: creating jobs, offering entrepreneurial opportunities, and increasing the profitability of software companies. The findings alleviated fears connected to using automation and AI in software development and encourage further deployment of AI-technologies.

Keywords

Software development, software development life cycle, machine learning, artificial intelligence, generative pre-trained transformers, ChatGPT, natural language processing, deep learning, automation.

CR Categories (ACM Computing Classification System, 1998 version): D.2.2, D.2.3, F.3.1, I.2.0, I.2.2, I.2.7, J.6, K.7.1

List of Abbreviations

AI	Broad definition of AI is intelligent or rational behavior demonstrated by machines, powered by algorithms.
DL	Deep learning is a subset of machine learning, which utilizes neural networks for problem solving.
ChatGPT	Chatbot developed by OpenAI based on large language models.
GPT	Generative pre-trained transformers are a large language models-based form of generative artificial intelligence, which generates responses based on the prompts it is given.
LLMs	Large language models are machine learning algorithms capable of comprehending and producing human language.
ML	Machine learning is a type of artificial intelligence, capable of learning from data and making predictions on new data.
NLP	Natural language processing combines machine learning and statistical models with computational modeling of natural language, enabling machines to comprehend and generate human language.
SDLC	Software Development Life Cycle is the set of phases and processes used in the development of software.
UBS	Formerly known as Union Bank of Switzerland, today known as UBS Group AG. UBS is a multinational corporation, providing financial services and acting as an investment bank.

Table of Contents

1	INTRODUCTION.....	1
1.1	Overview of Automation and Machine Intelligence in Software Development	1
1.2	Research Problem and Research Aim	3
1.3	Research Gap.....	3
1.4	Research Questions.....	4
1.5	Thesis Structure.....	5
2	METHODS.....	6
3	RESULTS – Bibliometric Analysis	14
3.1	Yearly Scientific Production Globally.....	14
3.2	Average and Total Article Citations per Year	16
3.3	Countries with the Highest Citation Numbers	18
3.4	Country Collaboration Networks	19
3.5	Most Productive Authors	21
3.6	Most Cited Authors	22
3.7	Ten Most Cited Papers.....	23
3.8	Most Frequent Author’s Keywords	24
3.9	Words Extracted from Abstracts	26
3.10	Clusters Formed from Author’s Keywords	28
4	RESULTS – Selective Literature Synthesis and Analysis.....	29
4.1	Findings Relating to All Phases of SDLC.....	37
4.1.1	Efficiency	38
4.1.2	Generalizability.....	38
4.1.3	Limitations.....	39
4.1.4	Risks.....	41
4.2	Requirement Engineering and System Design	42
4.3	Code Writing	44
4.4	Testing, Quality Assurance, and Software Analysis.....	45
4.5	Debugging and Maintenance	47
4.6	Documentation	48

4.7	Software Engineering Research	49
5	DISCUSSION	50
5.1	Aims & Objectives	50
5.2	Data Collection	50
5.3	Bibliometric Analysis	51
5.3.1	Analysis of Productivity in the Dataset	51
5.3.2	Analysis of Dataset Content	52
5.4	Selective Literature Synthesis and Analysis	52
5.4.1	SWOT Analysis	54
5.5	Limitations.....	58
5.6	Validity and Reliability	58
5.7	Conclusion	59
5.8	Future Work	60
	REFERENCES	61

1 INTRODUCTION

1.1 Overview of Automation and Machine Intelligence in Software Development

Denning & Lewis (2019) discuss six levels of machine intelligence. Level 0 machine intelligence, which is not a form of machine intelligence, is automation where automatons replace human actions, such as assembly line robots. At level 1 there are rule-based systems created with if-else structures, such as chess engines. Level 2 is supervised learning, where machines learn with the help of a training agent and training data for giving proper outputs for the types of inputs they are given. At level 3 there is unsupervised learning, which can make improvements to its behavior without a training agent or training data. At level 4 is multiagent interactions, where machine intelligence emerges from a large collective of different code segments, each having a particular function. Also, human-computer co-operation is a type of level 4 machine intelligence. Level 5 machine intelligence is capable of creativity. At level 6, *artificial intelligence* (AI) has humanlike common sense and consciousness. (Denning & Lewis, 2019). A broad definition of AI is intelligent or rational behavior demonstrated by machines (Russell & Norvig, 2009, pp. 1-5), powered by algorithms. There is no certainty whether it is possible to build a level 6 machine intelligence (Denning & Lewis, 2019). Reaching level 6 machine intelligence would be a step towards *Artificial General Intelligence* (AGI).

New AI techniques, such as *large language models* (LLMs), *natural language processing* (NLP), *generative pre-trained transformers* (GPTs), *machine learning* (ML), and *deep learning* (DL), have emerged, offering new ways to perform SDLC tasks, providing new forms of automation and efficiency into the software development sector and all sectors at large. LLMs have been trained with a vast amount of data, providing them with a capacity for comprehending and producing human language (IBM, 2024). NLP combines machine learning and statistical models with computational modeling of natural language, enabling machines to comprehend and generate human language (IBM, 2024). GPTs are a type of neural network model used in generative AI applications, such as interactive chatbots (Amazon Web Services, 2024).

Machine learning is a subset of AI, which algorithmically imitates human learning, leading to improvements in accuracy (IBM, 2024). ML technologies were developed in the 90s, which started providing remarkable results. ML gains its functionality from being exposed to a large set of examples, instead of programmed rulesets. (Denning & Lewis, 2019).

Deep learning is a form of machine learning which has utilized neural networks, statistics, and applied math in its development (Goodfellow et al., 2016, pp. 8, 26). The modern wave of DL, which started around 2006, is powered by modern machine learning models, such as *multi-layer perceptrons* (Goodfellow et al., 2016, pp. 9, 14, 18). In the early 2010s (Denning & Lewis, 2019) (Goodfellow et al., 2016, p. 26), DL became increasingly more useful and popular as a result of faster computing, larger datasets, and improved model training techniques (Goodfellow et al., 2016, p. 26).

What should we think about these new emerging AI technologies? Are they a threat or an opportunity? These technologies are powerful tools, creating efficiency and automation benefits to companies and society at large. Does their ability to automate processes create job displacement effects and take our jobs? According to Russell & Norvig (2009, p. 1034), information technology related automation, and especially AI, has produced more jobs than they have displaced, and the jobs they have created are more interesting and high paying than the jobs they have replaced. Despite the risks, these technologies offer great benefits, and by learning to utilize them, we can maximize the benefits we receive from them.

In 1997 World Chess Champion Garry Kasparov lost a chess match to the chess Computer 'Deep Blue' (Denning & Lewis, 2019). As a result of his defeat, Kasparov found a new approach to playing chess: human co-operation with AI, which forms a stronger unit than a human or an AI alone can form (Denning & Lewis, 2019). The current forms of AI are 'intelligent agents' designed to assist humans, increasing human efficiency and productivity, while in the past, AI was used as 'expert systems', with the goal of replacing human labor (Russell & Norvig, 2009, p. 1034). Human ingenuity enhanced by AI assistance is a powerful combination, and it will be interesting to see what kind of advancements these technologies will bring to software development in the future.

1.2 Research Problem and Research Aim

The topic of AI and automation in software development are central and crucial topics, but as topics they are extensive, and furthering our understanding and providing to the existing body of knowledge will assist in enhancing software development and finding the best ways for utilizing these technologies. The aim of this paper is to map out in which SDLC processes automation and AI can be applied in, identify the strengths and weaknesses of automation and AI for performing SDLC tasks, and recognize the consequences entailing from utilizing automation and AI in software development.

1.3 Research Gap

For analyzing the research gap regarding the selective literature synthesis, there were six articles identified, discussing the use of AI in different SDLC phases. Barenkamp et al.'s (2020) study focused on mapping out the applications of AI in different SDLC phases, while the results, according to the authors, fell short of being comprehensive due to the topic of AI in SDLC being too broad. Barenkamp et al. (2020) discussed articles by Sorte et al. (2015) and Padmanaban & Sharma (2019), stating these articles were incomprehensive, lacking in relevant sources, and were found to be explorative. Smith & Wei's (2024) paper did not have comprehensive sources, having only 18 references, and the results were descriptive and lacking in actionable insights. Sofian et al. (2022) and Singh & Sambhav's (2023) studies were systematic mapping studies, goals of which are to provide an overview of a field of research by means of classification (Sofian et al., 2022), exploring the field, but lacking in depth of analysis of the topic.

The topic of AI in software engineering is extensive (Barenkamp et al, 2020), and all additional research in this topic will assist in furthering discussion and understanding of the field. Selective literature synthesis and analysis done in this paper discusses different SDLC phases and processes automation and AI can be applied to, and analyzes advantages, disadvantages, and consequences that follow from this. Recognizing advantages and disadvantages, we can identify in which contexts automation and AI is most applicable to,

and identify risks and limitations, giving us the capacity to find ways to overcome them and make informed decisions.

Regarding the bibliometric analysis, after searching databases of Scopus, Google Scholar, JSTOR, IEEE, and Web of Science, there were no similar bibliometric analyses found. One study was identified by Boukhlif et al. (2023), which performed a bibliometric analysis limited to the topic of AI and software testing, while the bibliometric analysis performed in this paper covers all the phases of SDLC.

1.4 Research Questions

Four research questions (RQ) in total were created. One research question was connected to bibliometric analysis (RQ1), and three research questions relate to selective literature synthesis and analysis (RQ2-4). The four research questions can be seen below.

Bibliometric analysis

RQ1: What is the current state of research on automation and AI in software development? Identifying themes, trends, and contributors to research.

Selective literature synthesis and analysis

RQ2: What SDLC phases and processes are AI and automation applicable to?

RQ3: What are the advantages and disadvantages of applying different AI and automation techniques to SDLC processes?

RQ4: What are the problems, solutions to problems, and opportunities presented by applying automation and AI to SDLC processes?

1.5 Thesis Structure

This paper follows IMRAD-structure: Introduction, Methods, Results, and Discussion. The RQ1 is answered in the Results chapter addressing the bibliometric analysis, which is summarized in the Discussion chapter of this paper. Data for answering RQ2-4 is gathered in the Results chapter dealing with the selective literature synthesis and analysis, and the data is refined and further analyzed in the Discussion chapter into a form which answers RQ2-4. The SWOT analysis in the Discussion chapter is aimed at helping answer RQ3 and it specifically answers RQ4.

This paper leaves out of discussion automation techniques such as low-code or no-code application creation tools; DevOps techniques such as continuous integration and continuous delivery; machine vision; and robotic process automation, which utilizes software robots, capable of interacting with software applications, to perform actions imitating human activities (Syed et al., 2020). This paper did not specifically include an article in the literature synthesis and analysis discussing graphical user interface design; however, there was a finding stating that ChatGPT can be used for user interface generation (Fraïwan & Khasawneh, 2023).

2 METHODS

This paper has two separate research phases and research methodologies. Firstly, a bibliometric analysis was conducted to map out the current state of research in automation and AI technologies for automating SDLC processes. Secondly, a selective literature synthesis and analysis was conducted to examine the current and future state of automation and AI technologies assisting in SDLC processes.

In data collection, both research phases followed the PRISMA guidelines (Page et al., 2021) to ensure good data collection practice were followed and that the steps taken in the data collection were objective and transparent. The bibliometric analysis utilized the Biblioshiny application found in the Bibliometrix R package for generating insights from the dataset collected from Scopus-database. Both research phases utilized thematic analysis as an analysis method because it is apt for analyzing data, allowing recognition of trends, themes, and patterns, and it emphasizes the interpretation of data as a part of the analysis process (Castleberry & Nolen, 2018).

The thematic analysis framework proposed by Castleberry & Nolen (2018) was utilized. The framework consists of five steps: compiling, disassembling, reassembling, interpreting, and concluding. Compiling refers to collecting and arranging your data. In disassembling the gathered data is broken down into small components by recognizing the notable elements of the data. Reassembling means reorganizing the disassembled elements into an overarching theme. Interpreting is done simultaneously with the earlier phases, where the researcher identifies implications from the data. Concluding answers the research questions of the study. (Castleberry & Nolen, 2018.)

To gather materials for conducting the bibliometric analysis, documents were searched on Scopus database with the following search phrase:

TITLE-ABS-KEY(((automation OR automating OR automated) OR (artificial AND intelligence) OR (ai) OR ('machine learning') OR {ml} OR (large AND language AND model*) OR (computer AND aided AND software AND engineer*)) AND ({software development} OR ('software development life cycle') OR 'sdlc'))AND
PUBYEAR > 2009 AND PUBYEAR < 2024 AND (LIMIT-TO (SRCTYPE,"j") OR LIMIT-TO (SRCTYPE,"p")) AND

(LIMIT-TO (SUBJAREA,"COMP")) AND (LIMIT-TO (LANGUAGE,"English")) AND (EXCLUDE (EXACTKEYWORD,"Software Design"))

Table 1 clarifies the search phrase and the structure used in formation of the search phrase. The search phrase limited the source type to 'conference proceedings' or 'journal articles.' Using the search phrase, the subject area was limited to 'Computer Science' to reduce the number of results which were outside the scope of this research. The search phrase also excluded the keyword 'software design', because the keyword increased the search results from 1049 to 5246, and after examination, it was determined that the added 4197 search results diluted the quality of the search results significantly, a great portion of the articles being outside of the scope of this study. After removing the keyword 'software design', there were still keywords in the search results discussing similar topics, such as: 'design (14)', 'computer aided design (8)', and 'design and implementations (5).'

Table 1. Topics in the search phrase segmented into search terms.

Topic and Cluster	Search Terms
automation	(automation OR automating OR automated) OR (artificial AND intelligence) or ai OR (‘machine learning’) OR {ml} OR (large AND language AND model*) OR (computer AND aided AND software AND engineer*) AND
software development	{software development} OR (‘software development life cycle’) OR 'sdlc'

The search was done on 28th of February 2024, and the results included 1049 articles. The year range of the analysis was aimed to be between 10 to 15 years in order to keep the articles of analysis recent, and to avoid making the pool of articles found using the search phrase too large. To include the years between 2010 and 2023 was a compromise between 10 and 15 years, and luckily, this year range included the breakthrough in popularity of deep learning, which began in the early 2010s (Denning & Lewis, 2019) (Good fellow et al., 2016, p.26). The year 2024 was left out of examination due to the year not having been completed yet, and therefore the results are not relatively comparable to other years in the chosen period, hence including the year 2024 would complicate the analysis of the findings. The articles found with the search phrase were manually inspected, looking for articles matching the inclusion criteria, inspecting the titles, abstracts, keywords, and in some cases the full texts.

In its entirety, the inclusion criteria included the following conditions:

1. The topic must deal with automation or AI in the context of SDLC processes.
2. The focus was on the software development sector, and topics outside of this scope, such as using ML techniques to automate processes in other industries, were not included.
3. Years from 2010 to 2023.
4. In English.
5. Journal articles or conference papers.

In the end, of the 1049 articles, 480 documents were gathered for the purpose of performing a bibliometric analysis. The overall search strategy is outlined in Figure 1. Details regarding the search results can be examined more thoroughly from Table 2.

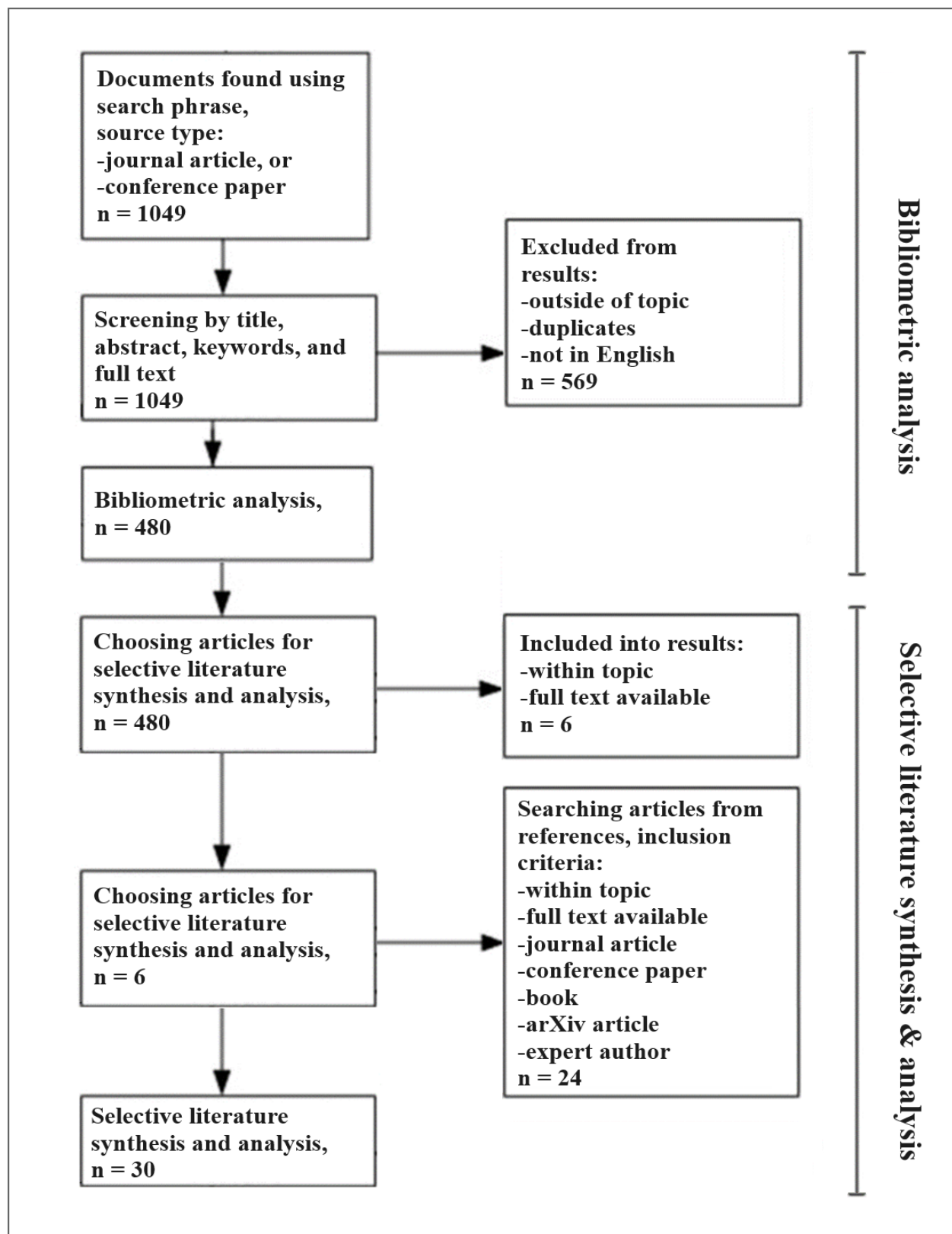


Figure 1. Document search strategy used in the bibliometric analysis and the selective literature synthesis and analysis.

Table 2. Information about the bibliometric dataset.

Description	Results
Timespan	2010–2023
Documents	480
Sources (journals, books, etc)	245
Annual growth rate %	18,79
Average citations per document	11,08
References	14498
Author's keywords (DE)	1478
Keywords plus (ID)	1755
Authors	1431
Authors of single-authored docs	33
Co-authors per document	3,23
International co-authorships %	17,29
<i>Document Types</i>	
article	289
conference paper	176
conference review	1
editorial	1
note	1
review	12

For conducting a selective literature synthesis and analysis, 6 articles were chosen from the 480 articles chosen for the bibliometric analysis, with the same inclusion criteria as was used in choosing the articles for the bibliometric analysis, with the addition that the chosen articles had full texts available. The initial 6 articles were chosen in a manner which would find a set of articles covering all SDLC phases, such as requirements engineering, code writing, testing, debugging, software maintenance, and documentation.

After having chosen the initial 6 articles, more articles were sought using the reference lists of the initial 6 articles, using the following inclusion criteria:

1. Within topic.
2. In English.
3. Full text available.
4. Journal article, conference paper, book, arXiv article, or expert author.

During this process, articles were chosen in a manner which would evenly cover all topics, phases, and processes of SDLC, making sure no important topic would be left without findings. In this manner, an additional 24 articles were gathered. A total of 30 articles were chosen for selective literature synthesis and analysis. The search strategy is described in Figure 1. Table 3 shows the chosen articles and their document types.

Table 3. Information about the articles chosen for the selective literature synthesis and analysis.

Author	Article title & article type	Year
Journal article		
Ahmed et al.	CaPBug-A Framework for Automatic Bug Categorization and Prioritization Using NLP and Machine Learning Algorithms.	2021
Bale et al.	ChatGPT in Software Development: Methods and Cross-Domain Applications.	2023
Boukhelif et al.	A Decade of Intelligent Software Testing Research: A Bibliometric Analysis.	2023
Dwivedi et al.	Opinion Paper: “So what if ChatGPT wrote it?” Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy.	2023
Ebbah, J. O. G.	Deploying Artificial Intelligence Techniques in Software Engineering.	2002
Garouis et al.	NLP-assisted software testing: A systematic mapping of the literature.	2020

Garuois & Mäntylä	When and what to automate in software testing? A multi-vocal literature review.	2016
Gill & Kaur	ChatGPT: Vision and challenges.	2023
Jayavel et al.	Use of Artificial Intelligence in Automation of Sequential Steps of Software Development / Production.	2013
Khomh et al.	Software Engineering for Machine-Learning Applications: The Road Ahead.	2018
Man-Fai et al.	Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review.	2023
Shafiq et al.	A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.	2021
Shankar & Chaudhari	Framework for the Automation of SDLC Phases using Artificial Intelligence and Machine Learning Techniques.	2023
Surameery & Shakor	Use Chat GPT to Solve Programming Bugs.	2022
Wan et al.	How does Machine Learning Change Software Development Practices?	2021
Wiklund et al.	Impediments for software test automation: A systematic literature review.	2017
Book		
Partridge, D.	Artificial Intelligence and Software Engineering, Understanding the Promise of the Future.	1998
Conference paper		
Amaro et al.	AI Unreliable Answers: A Case Study on ChatGPT.	2023
Di Stefano & Menzies	Machine learning for software engineering: case studies in software reuse.	2002
Iyer et al.	Mapping Language to Code in Programmatic Context.	2018
Panichella & Ruiz	Requirements-Collector: Automating Requirements Specification from Elicitation Sessions and User Feedback.	2020

Raza, F. N.	Artificial Intelligence Techniques in Software Engineering (AITSE).	2009
Ross et al.	The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development.	2023
Surana et al.	Intelligent Chatbot for Requirements Elicitation and Classification.	2019
Vogelsang & Borg	Requirements Engineering for Machine Learning: Perspectives from Data Scientists.	2019
Yahav, E.	Programming with "Big Code".	2015
Zhang, D.	Applying machine learning algorithms in software development.	2000
arXiv article		
Fraiwan & Khasawneh	A Review of ChatGPT Applications in Education, Marketing, Software Engineering, and Healthcare: Benefits, Drawbacks, and Research Directions.	2023
Ma et al.	The Scope of ChatGPT in Software Engineering: A Thorough Investigation.	2023
Expert article		
Dijkstra, E. W.	A Preliminary Investigation into Computer Assisted Programming.	1967 - 1970

3 RESULTS – Bibliometric Analysis

3.1 Yearly Scientific Production Globally

Figure 2 shows the yearly number of articles produced between the years 2010 and 2023. Production peaks in the year 2012, which declines in the years 2013 and 2014, while the production numbers remain relatively high. In the year 2015, production has a strong decline, and the production remains stagnant in the years 2016 and 2017. Starting from the year 2018 and continuing to the year 2023, production rises, and in the year 2023, production reaches a new peak, exceeding production numbers of the year 2012.

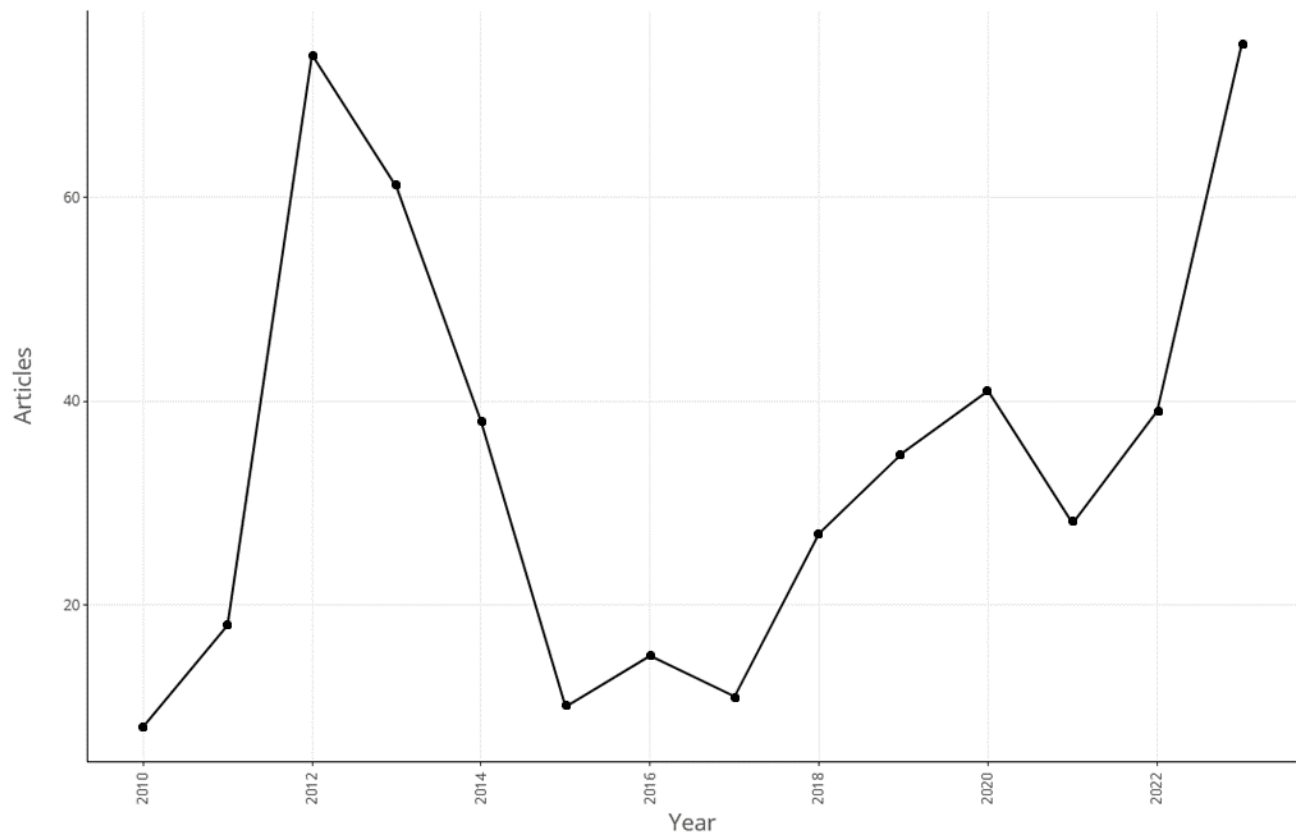


Figure 2. Yearly number of articles produced in the years between 2010 and 2023.

Figure 3 shows production numbers of the ten countries with the highest frequencies of scientific production. The numbers were divided into two periods: 2010 to 2014 and 2015 to 2023. This division was done to facilitate analysis regarding the reasons for the big surge in production in the years between 2010 and 2014, the stagnation in 2015 to 2017, and the resurgence of production starting in the year 2018 and continuing to the year 2023.

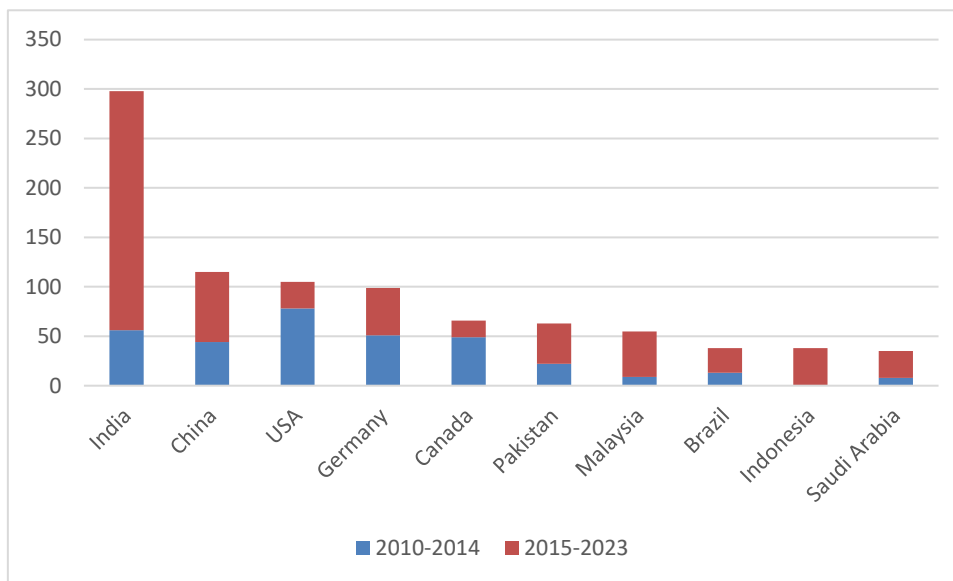


Figure 3. The ten countries with the highest frequencies of scientific production in the data set. Production numbers have been divided into two periods: 2010 to 2014 and 2015 to 2023.

All countries, except the Western countries of United States, Canada, and Germany, increased their production of articles in the period between 2015 and 2023, compared to their previous productions in the 2010 to 2014 period, with India increasing its production 4-fold, and Germany maintaining its production. United States, who was the highest producer of articles in the period between 2010 and 2014, reduced its output of articles to one third in the 2015 to 2023 period from its previous production levels in the 2010 to 2014 period. The reasons for this can have many explanations. Firstly, United States could simply have lost its interest in the field, perhaps the budget for this type of research was cut. Secondly, United States might have outsourced its scientific production in this field to countries like India, Malaysia, Indonesia, or Saudi Arabia, all of which happen to be strategic allies of United States. Third possible reason

could be that United States kept researching the field, but changed the keywords it used, or slightly changed the area of research.

3.2 Average and Total Article Citations per Year

Figure 4 shows the average article citations per year. This number is derived by dividing average article citations with citable years. For example, the year 2010 has 15 citable years. The citable years were adjusted by having the year 2024 only count as $\frac{1}{4}$ of a year to make the numbers closer to year 2024 more representative of reality.

An additional custom-tailored chart, Figure 6, was created for analyzing the relationships between average article citations, total citations per year, and numbers of articles released each year, by overlapping the Figures 2, 4, and 5 into one chart, making analysis of the three phenomena easier. Figure 5 illustrates the total citations per year. The documents released earlier in time have had more time to accumulate citations than more recent articles.

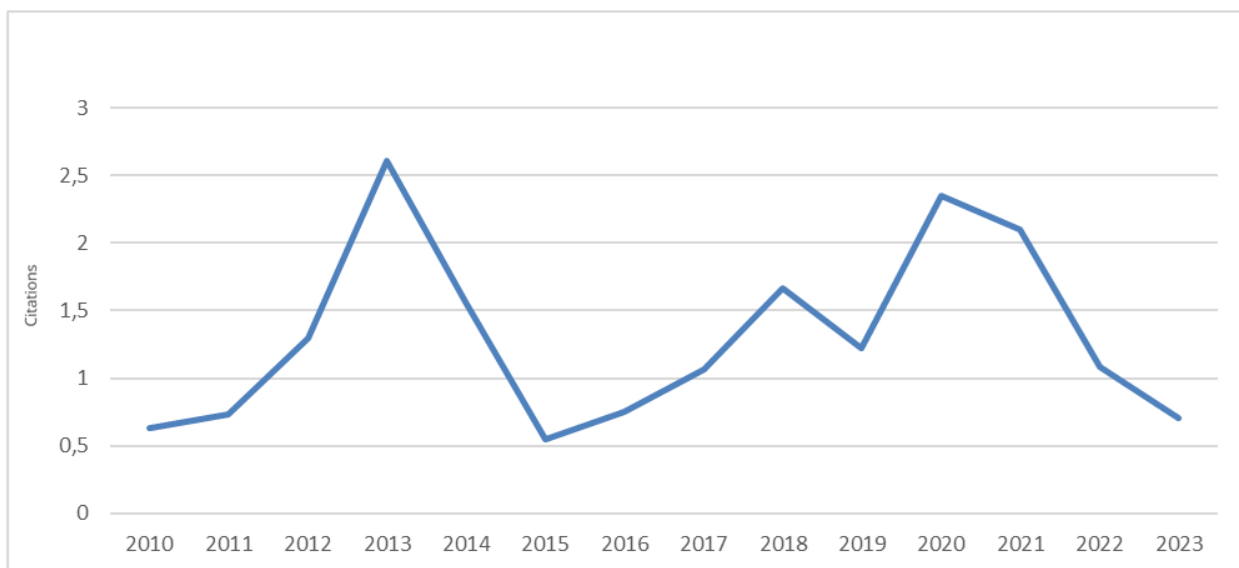


Figure 4. Average article citations per year between the years 2010 and 2023.

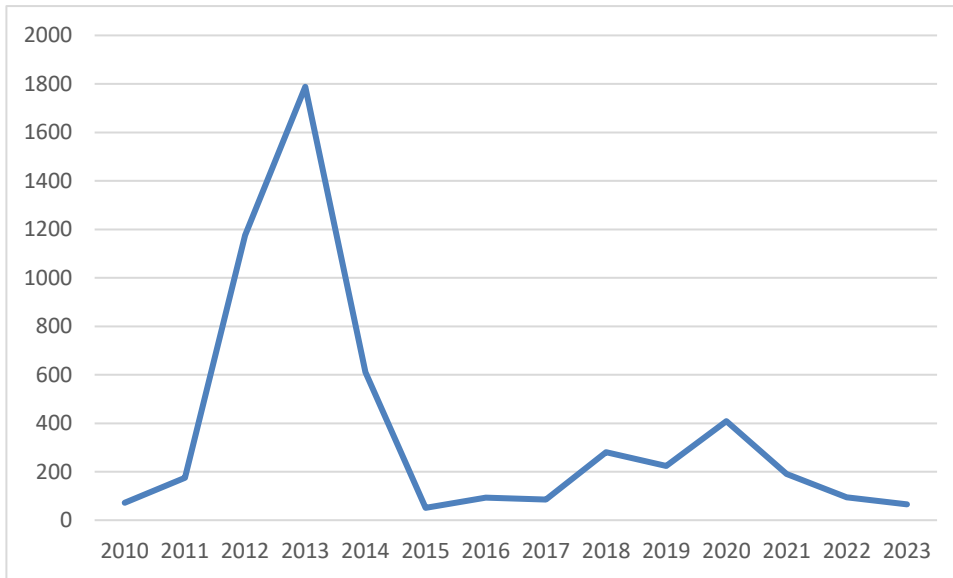


Figure 5. Total citations per year between the years 2010 and 2023.

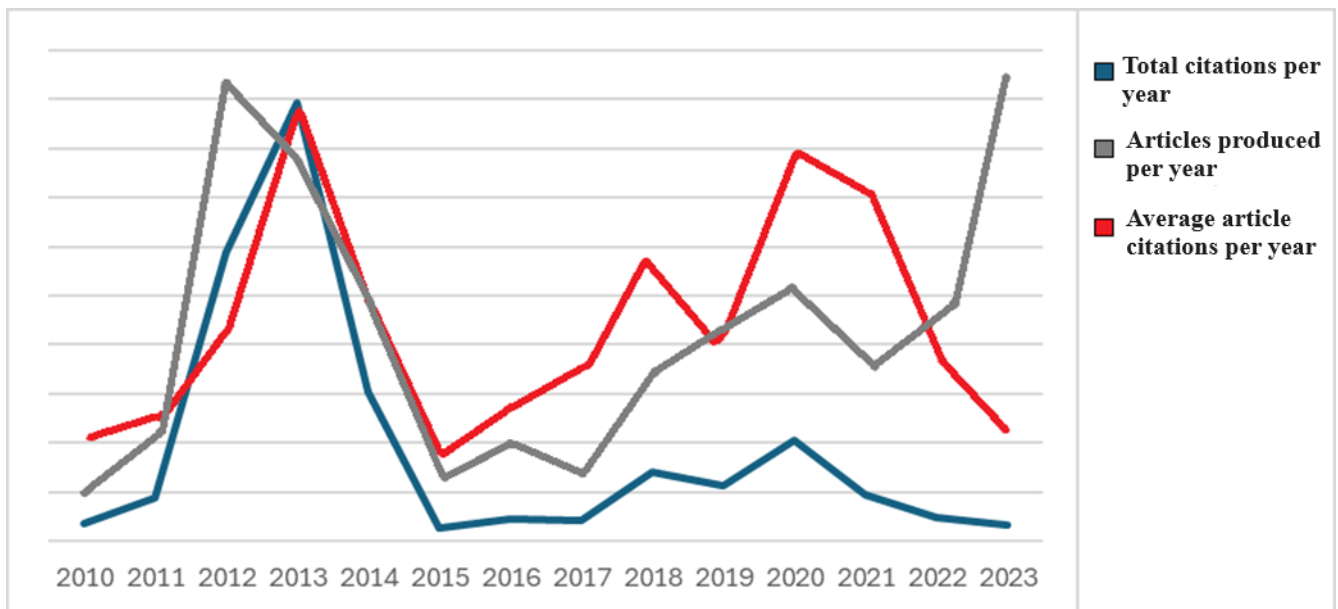


Figure 6. Overlapping of three graphs (Figures 2, 4, and 5): total citations, number of articles produced, and average article citations per year.

Observing Figure 6, the graph of average article citations per year follows closely the graph of articles produced per year. At years 2022 and 2023, the graphs don't follow each other: the article production surges, while citation numbers have declining trends, which is explained by the years 2022 and 2023 being

too recent to have accumulated citations yet. In this light, the years 2022 and 2023 can be good years, since they have high numbers of published articles, while the state of the citation numbers remains yet to be determined. The years from 2012 to 2014 were exceptionally good years, having high production of articles, high total citations, and high average citations. The years from 2018 to 2021 were good years as well, having moderately high numbers in all three graphs. These high citation numbers in the years 2012, 2013, and 2014 can be explained by many popular articles being published in these years. As can be seen later from Table 9, nine out of the ten most cited articles were published in the years 2012, 2013, and 2014.

3.3 Countries with the Highest Citation Numbers

Table 4 shows the ten most cited countries, and Table 5 displays the ten countries with the highest average article citations. Canada and India had a significant lead compared to other countries in terms of total citation counts. Canada, India, Germany, Saudi Arabia, and China are all in the lists of countries that have both highest scientific productions (Figure 3) in addition to being in the list of countries with the highest amounts of total citations. Canada and the European countries of Belgium, Portugal, Spain, Italy, and Switzerland are countries which are in both lists regarding highest total citations and highest average citations. In the ten countries with the highest total citations, six countries are European, the rest being Canada, India, Saudi Arabia, and China. Also, in the highest average article citations, eight countries are European, other countries being Canada and Australia. The article production numbers of singular European countries are not high, yet their rankings in the lists of total and average citations are notable.

Table 4. Ten most cited countries.

Country	Total Citations	Articles Produced	Average Citations
Canada	604	75	8,05
India	551	304	1,81
Belgium	172	12	14,33
Germany	172	108	1,59

Portugal	154	15	10,27
Spain	154	35	4,40
Italy	136	24	5,67
Switzerland	125	22	5,68
Saudi Arabia	93	38	2,45
China	88	122	0,72

Table 5. Ten countries with the highest average article citations.

Country	Total Citations	Articles Produced	Average Citations
Belgium	172	12	14,33
Portugal	154	15	10,27
Canada	604	75	8,05
Norway	32	4	8,00
Greece	84	13	6,46
Switzerland	125	22	5,68
Italy	136	24	5,67
Australia	69	13	5,31
Spain	154	35	4,40
United Kingdom	82	22	3,73

3.4 Country Collaboration Networks

Looking at Figure 7, the Louvain clustering algorithm identified 5 different country collaboration clusters from the data. Red cluster included India, United States, China, Norway, Singapore, Canada, and Australia. Besides Norway, the countries in this cluster included North American and Asia-Pacific countries. Blue

cluster has the European countries of Germany, Netherlands, and Italy, in addition to the South American country of Brazil. The green cluster included the Middle Eastern countries of Saudi Arabia, United Arab Emirates, and Egypt, in addition to countries of Pakistan and South-Korea. The yellow cluster consists of the Middle Eastern countries of Jordan and Bahrain. In the lilac cluster was included the following countries from Southeast Asia and Europe: Malaysia, Indonesia, United Kingdom, and Spain.

In the red cluster, countries with especially frequent collaboration with each other were United States and India, United States and China, as well as United States and Canada. In the lilac cluster, Malaysia and Indonesia had frequent collaboration with each other, also United Kingdom and Spain had frequent collaboration. In the green cluster, Saudi Arabia was a central figure, having frequent collaboration with United Arab Emirates, Egypt, and Pakistan. United States, Saudi Arabia, and Italy were the three countries with the most versatile collaboration networks, all three having collaboration with five different countries.

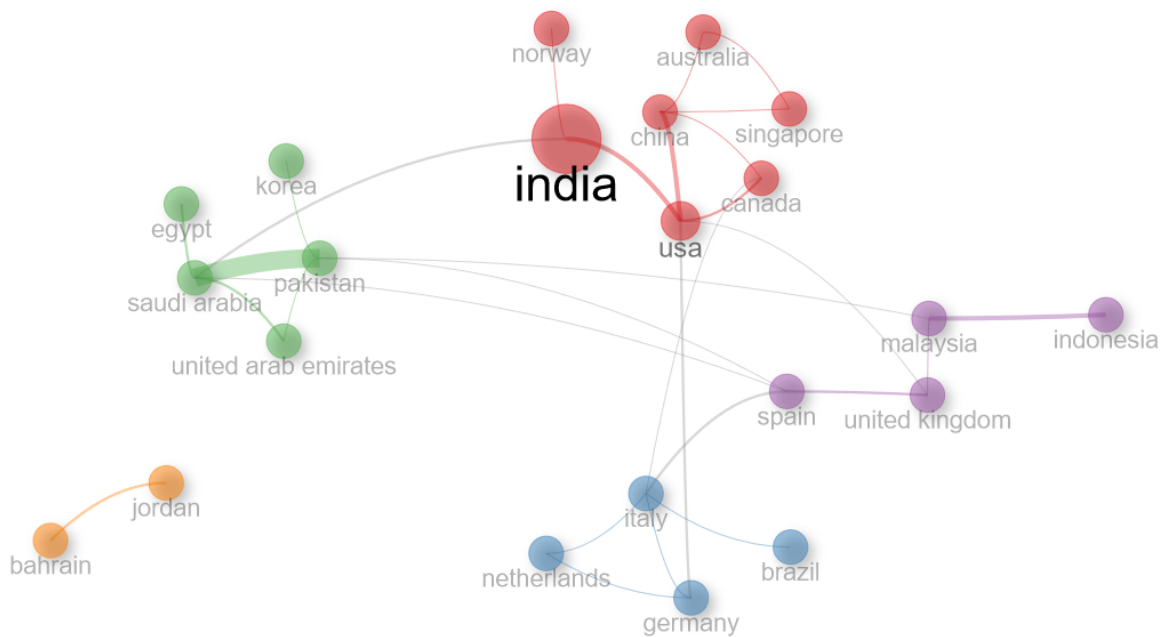


Figure 7. Clusters of collaboration networks between countries, thickness of connection-line represents the amount of collaboration between the two countries.

3.5 Most Productive Authors

Table 6 was custom-tailored to include only articles which had citations, and each author was manually verified to have the number of articles shown in the table. Four out of the ten authors in the list were Canadians, and only Pakistan was another country with more than one author from the same country. Canadian authors in this list focused on topics of automation, different AI techniques, and automated testing. European authors were connected to keywords addressing ML, test automation, and requirement engineering. Authors from India and Pakistan were connected to keywords pertaining to defect prediction, ML, and effort estimation. As an overall theme, testing, automation, and different AI techniques were themes focused on by the authors.

Table 6. Ten authors with the most articles published in the dataset.

Authors	Articles	Location	Keywords
Rhmann, Wasiur	4	India	Defect prediction; effort estimation
Adams, Bram	3	Canada	Testing; automated defect detection
Aftab, Shabib	3	Pakistan	Defect prediction; machine learning
Chechik, Marsha	3	Canada	Model transformation; automation
Salay, Rick	3	Canada	Deep learning; artificial intelligence
Godfrey, Michael W.	2	Canada	Machine learning; deep learning
Ionescu, Vlad Sebastian	2	Romania	Natural language processing; machine learning
Matloob, Faseeha	2	Pakistan	Defect prediction; machine learning
Straszak, Tomasz	2	Poland	Test automation; requirements specification
Wiklund, Kristian	2	Sweden	Automated testing; automation

3.6 Most Cited Authors

Table 7 shows the ten most cited authors in the dataset. The articles published by the most cited authors discussed topics around automated testing, fault prediction, automated debugging, and automated code refactoring, highlighting the significance of these topics within the research community. A lot of the authors and articles in this list had co-authorship on the popular articles, which caused there to be a lot of authors from same countries, such as Singapore, Portugal, and Belgium. In addition to the aforementioned countries, there were authors from United States and Canada.

Table 7. Most cited authors in the dataset.

Author	Location	Total	
		Citations	Articles in the Dataset
Chandra, S.	USA	468	SemFix: Program repair via semantic analysis. (2013).
Nguyen, HDT	Singapore	468	SemFix: Program repair via semantic analysis. (2013).
Qi, D.	Singapore	468	SemFix: Program repair via semantic analysis. (2013).
Roychodhury, A.	Singapore	468	SemFix: Program repair via semantic analysis. (2013). Gzoltar: An eclipse plug-in for testing and debugging. (2012).
Campos, J.	Portugal	235	Using HTML5 visualizations in software fault localization. (2013).
Abreu, R.	Portugal	235	Gzoltar: An eclipse plug-in for testing and debugging. (2012). Using HTML5 visualizations in software fault localization. (2013).
Baesens, B.	Belgium	189	Toward comprehensible software fault prediction models using bayesian classifiers. (2013).

			Investigating associative classification for software fault prediction. (2014).
Dejaeger, K.	Belgium	168	Toward comprehensible software fault prediction models using bayesian classifiers. (2013).
Verbraken, T.	Belgium	168	Toward comprehensible software fault prediction models using bayesian classifiers. (2013).
			Identification and application of Extract Class refactorings in object-oriented systems. (2012).
Fokaefs, M.	Canada	145	Software Engineering for Machine-Learning Applications: The Road Ahead. (2018).

3.7 Ten Most Cited Papers

In Table 8 have been outlined the ten most cited papers in the dataset. Nine out of the ten papers were from the 2010 to 2014 period, while only one was from the 2015 to 2023 period. Perhaps this was because the articles from the 2010 to 2014 period have had more time to accumulate citations. Nine out of the ten articles discussed topics of fixing bugs, detecting bugs, quality assurance, or test case prioritization, and one article discussed requirements prioritization. From this could be deduced that AI techniques are particularly applicable in these tasks, or there is a high interest in finding ways to automate these processes.

Table 8. Ten most cited papers in the dataset.

Paper (Total Citations / Total Citations per Year)
1. Nguyen et al. (2013). SemFix: Program repair via semantic analysis. (468 / 39,00)
2. Dejaeger et al. (2013). Toward Comprehensible Software Fault Prediction Models Using Bayesian Network Classifiers. (168 / 14,00)
3. Campos et al. (2012). GZoltar: an eclipse plug-in for testing and debugging. (143 / 11,00)
4. Perini et al. (2013). A Machine Learning Approach to Software Requirements Prioritization. (125 / 10,42)
5. Thomas et al. (2014). Static test case prioritization using topic models. (113 / 10,27)
6. Malhotra & Jain. (2012). Fault Prediction Using Statistical and Machine Learning Methods for Improving Software Quality. (112 / 8,62)
7. Giger et al. (2012). Method-level bug prediction. (112 / 8,62)
8. Maiga et al. (2012). SMURF: A SVM-based Incremental Anti-pattern Detection Approach. (102 / 7,85)
9. Chaturvedi K. K. & Singh V. B. (2012). Determining Bug severity using machine learning techniques. (93 / 7,15)
10. Hammouri et al. (2018). Software Bug Prediction using Machine Learning Approach. (92 / 13,14)

3.8 Most Frequent Author's Keywords

In Table 9 are shown the ten most frequent 'author's keywords' as well as 'keywords plus' found from the articles. Looking at the author's keywords, seven out of ten keywords are directly related to AI techniques or automation: 'machine learning', 'natural language processing', 'artificial intelligence', 'software defect prediction', 'automation', 'classification', and 'deep learning.' One third of the occurrences of ten most frequently used author's keywords consisted of the keyword 'machine learning, emphasizing the centrality

of this term in the research field. Software testing was the second most used author's keyword, showing there to be a large interest towards automating software testing processes.

Observing the 'keywords plus', four of them are related to testing and quality assurance related activities: 'software testing', 'testing', and 'software quality', and 'automation.' Four of the keywords plus are related to automation or machine intelligence techniques: 'automation', 'artificial intelligence', 'learning systems', and 'forecasting.' The word 'automation' was found in both author's keywords and keywords plus. During the process of choosing articles for this bibliometric analysis, the keyword 'automation' was noted to be strongly linked to articles discussing DevOps testing automation and agile development methods.

Table 9. Ten most frequent author's keywords and keywords plus.

Author's Keywords	Occurrences	Keywords Plus	Occurrences
machine learning	94	software engineering	98
software testing	36	software testing	42
software development	34	software development process	33
software engineering	31	automation	31
natural language processing	20	computer software selection and evaluation	24
artificial intelligence	18	artificial intelligence	21
software defect prediction	17	learning systems	21
automation	16	testing	17
classification	15	software quality	16
deep learning	15	forecasting	15

Figure 8 shows the ten most frequent author's keywords in streamgraph form. Observing Figure 8, the keywords 'machine learning', 'automation', and 'natural language processing' started to be used in this dataset since 2012. Keyword 'machine learning' has been on a steady rise since 2019, becoming the most popular keyword in the dataset. The keywords 'artificial intelligence', 'software defect prediction', and

‘classification’ started to be used in the dataset since 2013. The keyword ‘deep learning’ started to be used in the articles of this dataset since 2019.

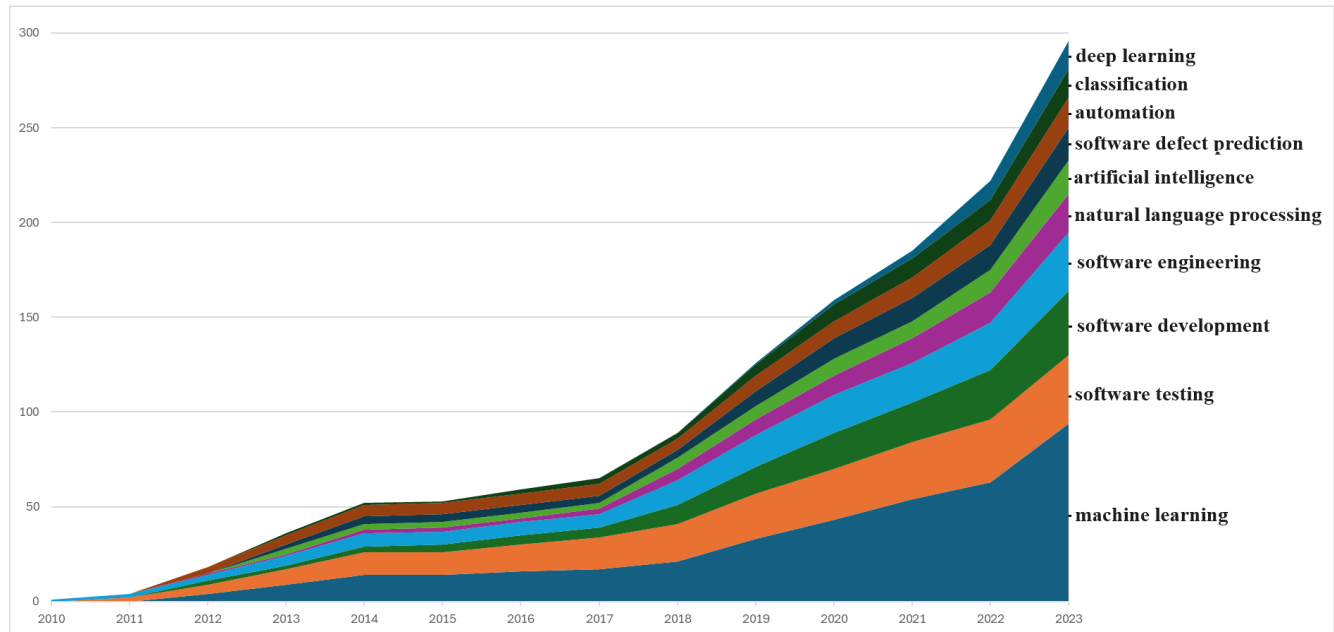


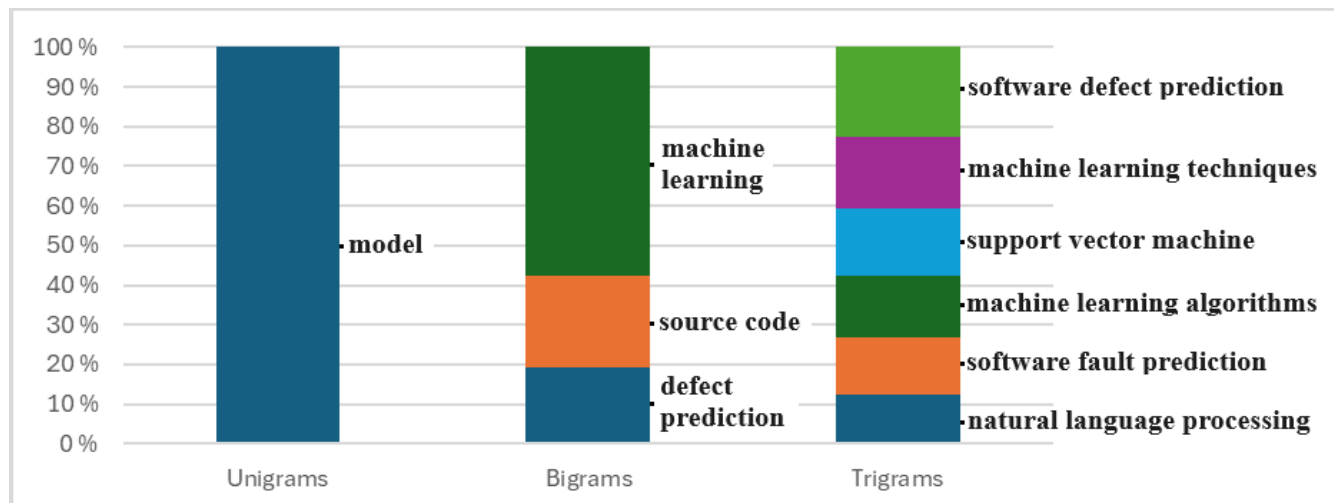
Figure 8. Streamgraph of ten most popular author's keywords from year 2010 to 2023.

3.9 Words Extracted from Abstracts

Table 10 has unigrams, bigrams, and trigrams of the most frequent words found from abstracts of the dataset articles. The most relevant unigrams, bigrams, and trigrams have been collected to Figure 9 to ease observation. In unigrams, the only word that could be directly connected to automation or machine learning is the word ‘model.’ In bigrams, there are two words that can be directly connected to automation: ‘machine learning’, ‘defect prediction’, and perhaps ‘source code’, because when performing defect prediction, one needs to analyze the source code. Trigrams directly related to automating software development processes are ‘software defect prediction’, ‘machine learning techniques’, ‘support vector machines’, which are ML algorithms used for classification tasks, ‘machine learning algorithms’, ‘software fault prediction’, and ‘natural language processing.’ Surprisingly, there was no mention of LLMs, which could be explained by the recency of the phenomenon.

Table 10. Ten most frequent unigrams, bigrams, and trigrams found from abstracts.

Unigram (occurrences)	Bigram (occurrences)	Trigram (occurrences)
software (1863)	software development (557)	software development process (78)
development (861)	machine learning (199)	development life cycle (49)
testing (559)	software testing (120)	software development life (45)
test (503)	development process (100)	software defect prediction (36)
model (403)	source code (80)	machine learning techniques (29)
code (364)	software engineering (77)	support vector machine (27)
process (356)	defect prediction (67)	machine learning algorithms (25)
approach (342)	software quality (57)	software fault prediction (23)
requirements (337)	life cycle (55)	natural language processing (20)
data (331)	software projects (50)	software development projects (20)

**Figure 9.** The most relevant unigrams, bigrams, and trigrams extracted from abstract words. The size of the stack in the column represents the frequency of the word(s) used.

3.10 Clusters Formed from Author's Keywords

Figure 10 shows 22 most frequently used author's keywords in four clusters, which were formed using Louvain clustering algorithm. Red cluster has words such as 'machine learning', 'software defect prediction', 'classification', 'deep learning', 'software metrics', 'software effort estimation', 'clustering', and 'feature selection.' These keywords are associated with using ML algorithms for giving predictions, recommendations, and classifications connected to software development processes. Observing Table 12, it can be verified that in the cases presented in the table, only ML was used for these kinds of applications. In the blue cluster are words connected to testing and quality assurance, such as 'software testing', 'automated testing', 'software quality', 'test automation', and 'genetic algorithm', which can be used for generating test cases. In the lilac cluster can be found words which are connected to requirements engineering, such as 'requirement engineering', 'natural language processing', and 'artificial intelligence.' NLP and AI can be used for automating the requirements elicitation process (Surana et al., 2019). Green cluster has words such as 'automation', 'devops', and 'effort estimation', which can be linked to agile software development techniques for automating and streamlining software development processes.

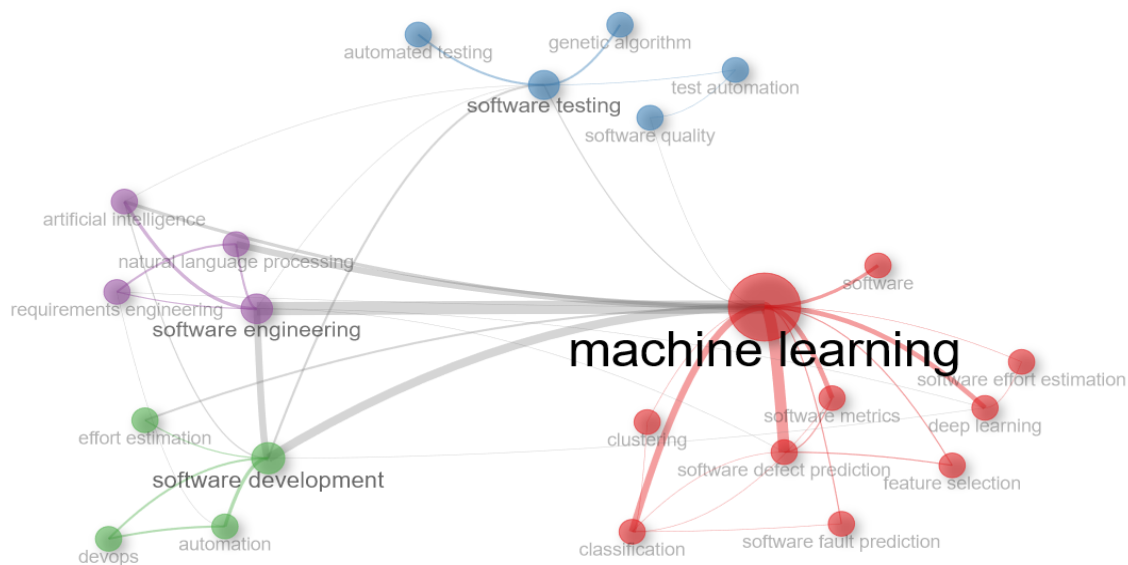


Figure 10. Co-occurrence network of author's keywords with four clusters.

4 RESULTS – Selective Literature Synthesis and Analysis

In the selective literature synthesis, 30 articles were synthesized and analyzed, looking for results regarding views on how automation and AI-based technologies will impact SDLC processes. The 30 articles chosen for analysis can be seen in Table 3. Positive results included increases in innovation, efficiency, adaptability, streamlining of processes, and reductions in costs. Critical findings discussed problems in reliability, interpretability, and security.

The findings included 52 positive findings and 16 findings which were critical towards using automation and AI to assist in software development processes. The findings were organized into 6 categories, which represent different phases of SDLC: ‘relating to all phases of SDLC’, ‘requirement engineering and system design’, ‘code writing’, ‘testing, quality assurance, and software analysis’, ‘debugging and software maintenance’, and ‘documentation.’ Also, a 7th category for ‘software engineering research’ was added, which does not necessarily relate to different phases of SDLC. The numbers regarding how the findings were distributed into different categories can be seen in Figure 11. The articles referenced for findings in each category are shown in Table 11. Table 12 summarizes the technologies and applications discussed in this chapter.

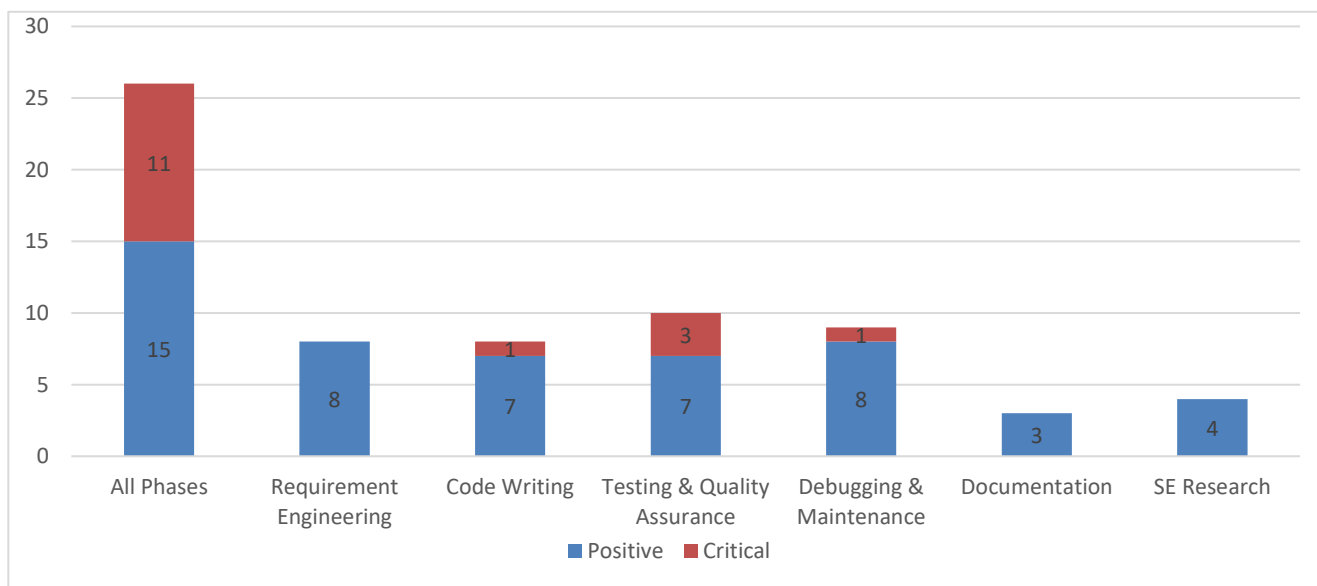


Figure 11. Numbers of all positive and critical findings by category.

Table 11. Articles referenced in each category for positive and critical findings.

SDLC STAGE	Type	Article
All SDLC Stages	Positive	
		Amaro et al. (2023). AI Unreliable Answers: A Case Study on ChatGPT.
		Bale et al. (2023). ChatGPT in Software Development: Methods and Cross-Domain Applications.
		Dwivedi et al. (2023). Opinion Paper: "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy.
		Ebbah, J. O. G. (2002). Deploying Artificial Intelligence Techniques in Software Engineering.
		Fraiwan & Khasawneh. (2023). A Review of ChatGPT Applications in Education, Marketing, Software Engineering, and Healthcare: Benefits, Drawbacks, and Research Directions.
		Jayavel et al. (2013). Use of Artificial Intelligence in Automation of Sequential Steps of Software Development / Production.
		Raza, F. N. (2009). Artificial Intelligence Techniques in Software Engineering (AITSE).
		Shankar & Chaudhari (2023). Framework for the Automation of SDLC Phases using Artificial Intelligence and Machine Learning Techniques.
		Zhang, D. (2000). Applying machine learning algorithms in software development.
	Critical	
		Amaro et al. (2023). AI Unreliable Answers: A Case Study on ChatGPT.
		Bale et al. (2023). ChatGPT in Software Development: Methods and Cross-Domain Applications.
		Dwivedi et al. (2023). Opinion Paper: "So what if ChatGPT wrote it?" Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy.
		Fraiwan & Khasawneh. (2023). A Review of ChatGPT Applications in Education, Marketing, Software Engineering, and Healthcare: Benefits, Drawbacks, and Research Directions.
		Gill & Kaur. (2023). ChatGPT: Vision and challenges.
		Ma et al. (2023). The Scope of ChatGPT in Software Engineering: A Thorough Investigation.
		Partridge, Derek. (1998). Artificial Intelligence and Software Engineering, Understanding the Promise of the Future.

Requirement Engineering & System Design	Positive	
		Bale et al. (2023). ChatGPT in Software Development: Methods and Cross-Domain Applications.
		Ebbah, J. O. G. (2002). Deploying Artificial Intelligence Techniques in Software Engineering.
		Fraiwan & Khasawneh. (2023). A Review of ChatGPT Applications in Education, Marketing, Software Engineering, and Healthcare: Benefits, Drawbacks, and Research Directions.
		Panichella & Ruiz. (2020). Requirements-Collector: Automating Requirements Specification from Elicitation Sessions and User Feedback.
		Shafiq et al. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.
		Surana et al. (2019). Intelligent Chatbot for Requirements Elicitation and Classification.
Code Writing	Positive	
		Ebbah, J. O. G. (2002). Deploying Artificial Intelligence Techniques in Software Engineering.
		Gill & Kaur. (2023). ChatGPT: Vision and challenges.
		Man-Fai et al. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review.
		Raza, F. N. (2009). Artificial Intelligence Techniques in Software Engineering (AITSE).
		Ross et al. (2023). The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development.
		Shafiq et al. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.
	Critical	
		Ma et al. (2023). The Scope of ChatGPT in Software Engineering: A Thorough Investigation.
Testing & Quality Assurance	Positive	
		Garouis et al. (2020). NLP-assisted software testing: A systematic mapping of the literature.
		Shafiq et al. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.

& Software Analysis	Critical	Wiklund et al. (2017). Impediments for software test automation: A systematic literature review.
		Ma et al. (2023). The Scope of ChatGPT in Software Engineering: A Thorough Investigation.
		Wiklund et al. (2017). Impediments for software test automation: A systematic literature review.
Debugging & Maintenance	Positive	
		Ahmed et al. (2021). CaPBug-A Framework for Automatic Bug Categorization and Prioritization Using NLP and Machine Learning Algorithms.
		Gill & Kaur. (2023). ChatGPT: Vision and challenges.
		Man-Fai et al. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review.
		Shafiq et al. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.
		Surameery & Shakor. (2022). Use Chat GPT to Solve Programming Bugs.
	Critical	Yahav, E. (2015). Programming with “Big Code”.
		Surameery & Shakor. (2022). Use Chat GPT to Solve Programming Bugs.
Documentation	Positive	
		Gill & Kaur. (2023). ChatGPT: Vision and challenges.
		Ma et al. (2023). The Scope of ChatGPT in Software Engineering: A Thorough Investigation.
		Man-Fai et al. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review.
SE Research	Positive	
		Bale et al. (2023). ChatGPT in Software Development: Methods and Cross-Domain Applications.
		Boukhelif et al. (2023). A Decade of Intelligent Software Testing Research: A Bibliometric Analysis.
		Man-Fai et al. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review.

	Shafiq et al. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages.

Table 12. Applications of automation and machine intelligence in SDLC processes mentioned in the literature synthesis.

SDLC Phase	Technology	Application
Requirement Engineering	ML	
		requirements modeling (Shafiq et al., 2021)
		requirements analysis (Shafiq et al., 2021)
		requirements selection (Shafiq et al., 2021)
		requirements prioritization (Shafiq et al., 2021)
		requirements classification (Shafiq et al., 2021)
		requirements tracing (Shafiq et al., 2021)
	NLP, ML, & AI	
		requirements elicitation chatbot (Surana et al., 2019)
	ML & DL	
		specification classification (Panichella & Ruiz, 2020)
		user feedback classification (Panichella & Ruiz, 2020)
	ChatGPT	
		user requirement analysis (Fraïwan & Khasawneh, 2023)
System Design	ML	

Code Writing		design and modeling (Shafiq et al., 2021)
		design pattern prediction (Shafiq et al., 2021)
		development effort estimation (Shafiq et al., 2021)
		user interface generation (Fraïwan & Khasawneh, 2023)
	ML	
		code cloning (Shafiq et al., 2021)
		code localization (Shafiq et al., 2021)
		code refactoring (Shafiq et al., 2021)
	ChatGPT	code labeling (Shafiq et al., 2021)
		code generation (Gill & Kaur, 2023)
	NLP	optimizing algorithms (Gill & Kaur, 2023)
		meta programming (Ebbah, 2002)
	LLM	
		code generation (Man-Fai et al., 2023)
		offering code completion suggestions (Man-Fai et al., 2023)
		translating code to other languages (Man-Fai et al., 2023)
		refining code (Man-Fai et al., 2023)
		clone detection (Man-Fai et al., 2023)
		defect detection (Man-Fai et al., 2023)
	lisp	explaining code (Ross et al., 2023)

		prototype generation (Ebbah, 2002)
Testing	ML	
		generating test cases (Shafiq et al., 2021)
		generating test data (Shafiq et al., 2021)
		test oracle generation (Shafiq et al., 2021)
		test effort estimation (Shafiq et al., 2021)
	NLP	
		generating test cases (Garouis et al., 2020)
	Scripts	
		automated testing (Wiklund et al., 2017)
Quality Assurance	ML	
		fault prediction (Shafiq et al., 2021)
		bug prediction (Shafiq et al., 2021)
		defect prediction (Shafiq et al., 2021)
	ML	
		software analysis (Shafiq et al., 2021)
		technique assessment (Shafiq et al., 2021)
		software process assessment (Shafiq et al., 2021)
		verification and validation (Shafiq et al., 2021)
Debugging	LLM	

Maintenance	ChatGPT	code refinement (Man-Fai et al., 2023)
		defect detection (Man-Fai et al., 2023)
		clone detection (Man-Fai et al., 2023)
		debugging (Gill & Kaur, 2023) (Surameery & Shakor, 2023)
		predicting bugs (Surameery & Shakor, 2023)
		explaining bugs (Surameery & Shakor, 2023)
	ML	
		predicting software maintainability (Shafiq et al., 2021)
		detecting aging software (Shafiq et al., 2021)
		estimating maintenance effort (Shafiq et al., 2021)
	Clustering (ML)	bug reporting: category prediction (Ahmed et al, 2021)
		bug reporting: priority prediction (Ahmed et al, 2021)
	Probabilistic models (ML)	
		reverse engineering code (Yahav, 2015)
		locating similar code (Yahav, 2015)
	ChatGPT	
		optimizing algorithms (Gill & Kaur, 2023)
Documentation	ChatGPT	
		describing code (Gill & Kaur, 2023)

	LLM	document generation (Ma et al., 2023)
		code summarization (Man-Fai et al., 2023)

4.1 Findings Relating to All Phases of SDLC

In this category of 'relating to all phases of SDLC' there were 15 positive and 11 critical findings. These findings were further divided into four subcategories: 'efficiency' with 8 positive findings; 'generalizability' with 7 positive findings; 'limitations' with 5 critical findings; and 'risks' with 6 critical findings. The numbers regarding how the findings of 'relating to all phases of SDLC' were distributed into different subcategories can be seen in Figure 12.

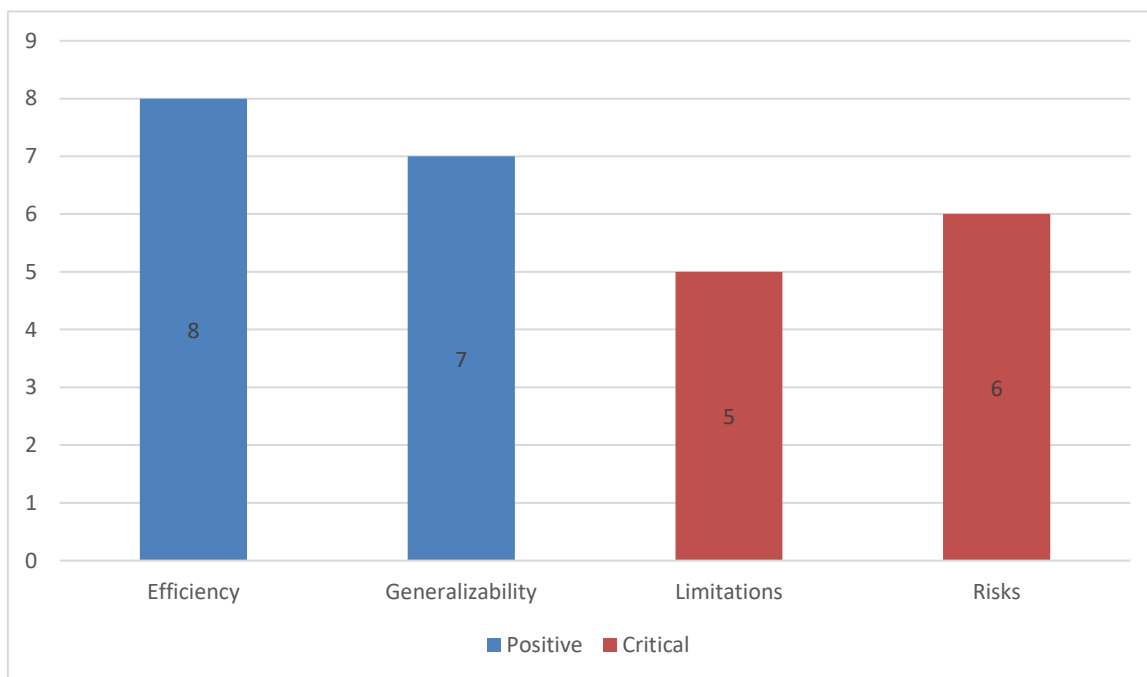


Figure 12. Findings by subcategory within the category of 'relating to all phases of SDLC.'

4.1.1 Efficiency

The term *efficiency* refers to the degree to which an AI technique can increase the efficiency and productivity of the software development phase or process, getting more outputs with less inputs, such as effort, time, or resources spent (Merriam-Webster, 2024). Themes discussed in this subcategory involve topics such as efficiency through cost and time reductions, increases in productivity, process streamlining, facilitation of innovation, and convenience. There were 8 positive findings categorized into this subcategory, which are summarized below.

Software engineering will unequivocally benefit from ML techniques (Zhang, 2000). Co-operation of humans with machine intelligence is a promising prospect for innovation, efficiency, and process streamlining (Bale et al., 2023). Utilizing ML and AI in different software development phases could lead to increases in efficiency, such as cost and time reductions (Shankar & Chaudhari, 2023). An AI-based risk assessment tool can be used to greatly reduce the resources spent towards assessing risks (Ebbah, 2002).

ChatGPT has capabilities for enhancing productivity in a range of different software development stages (Dwivedi et al., 2023). ChatGPT and other language models are powerful tools which can potentially revolutionize software engineering (Fraivan & Khasawneh, 2023). ChatGPT is occasionally unreliable, but users still appreciate the convenience and easy accessibility offered by chatbots (Amaro et al., 2023). It is interesting that the users appreciate ChatGPT despite the problems in reliability. Perhaps this shows that the users find they can reliably recognize when ChatGPT is giving unreliable answers, or perhaps the users verify the answers given by ChatGPT, removing the risk of inaccurate answers creating defects in their work. Overall, the users appear to be finding the use of ChatGPT as a net positive.

4.1.2 Generalizability

The findings in this subcategory found AI techniques to be highly generalizable, adaptive, and scalable. *Generalizability* refers to the applicability of a predefined sample to a broader population (Findley et al., 2021). In the context of AI techniques, generalizability refers to the ability of an AI technique to be used in

many different SDLC related contexts. *Adaptability* refers to the AI technique's ability to adjust to new conditions (Cambridge, 2024), or make the original application more adaptive, after being embedded with an AI algorithm (Zhang, 2000). *Scalability* refers to the AI technique's ability to improve the system it is applied to by providing new capacities, while the original system remains undisturbed (El-Rewini & Abd-El-Barr, 2005). A further definition for scalability is the capacity of the technology to perform while the workload grows, or the set of problems the tool is solving expands (Hayes, 2003). There were 7 positive findings classified into this subcategory, which are summarized below.

A considerable number of AI-based algorithms are highly generalizable and scalable to almost any software development process (Jayavel et al., 2013), and various AI algorithms are used in all phases of SDLC (Shankar & Chaudhari, 2023). ML is especially suitable for poorly understood problems, domains with large datasets, and domains requiring adaptability to a changing environment. ML algorithms can be used for building tools for tasks relating to maintaining and developing software. ML algorithms can also be embedded into software products to make them more autonomous and adaptive. (Zhang, 2000).

ChatGPT can be used in various software engineering tasks, such as code generation, debugging, testing, user requirement analysis, user interface generation, documentation, in addition to collaboration and knowledge sharing (Fraiwan & Khasawneh, 2023). ChatGPT was mentioned in every SDLC category and was found to have applications in all SDLC phases.

4.1.3 Limitations

This subcategory regarding *limitations* discusses findings talking about the lacks or boundaries to the capacities AI tools have. Recognizing lacks or limitations gives us the ability to find ways or additional resources to overcome them, which leads to enhancing the capabilities of the AI technologies. The findings of this section regarding limitations include topics such as unreliability, inaccuracy, as well as lack of interpretability and explainability. This subcategory included 5 critical findings, which are summarized below.

Partridge (1998) provides two criticisms for AI tools, but these findings are from the year 1998, and they do not necessarily represent the state of AI tools in the year 2024. Firstly, Partridge (1998, p. 252) says that when using AI tools for software development, it can be difficult to verify the developed software meets specifications. To provide context to this criticism, an AI tool does not necessarily follow all the procedures a human would follow, which ensure compliance with the requirements. Thus, it is left to the humans using the AI tools to spend extra effort verifying the results given by the AI tool. This view is supported by Amaro et al. (2023), who said in the context of ChatGPT, that in order to use it responsibly, its outputs should be interpreted and verified by a human, and users should be made aware of the limitations chatbots have. Gill & Kaur (2023) said that ChatGPT's functionality requires attention in correctness of facts, accuracy, and explainability. Ma et al. (2023) also discussed that ChatGPT's insufficient interpretability is a concern because of rigorous risk management and reliability requirements involved in software engineering. These criticisms, which coincide with Partridge's criticism from the year 1998, only concern the use of AI Chatbots, and they do not necessarily generalize to other AI tools.

In the second criticism Partridge (1998, p. 251) said that software developers can have high expectations for AI-based tools for supporting their workflow, but it is likely that these high expectations are not met, leading to disappointment. To provide clarity on what Partridge means, AI tools, such as ChatGPT, often produce results which might seem on surface level to be valid and accurate, but with closer inspection, it might turn out the outputs of the AI tool are incorrect and erroneous, creating a sense of disappointment and confusion for the user. In the context of LLM-based chatbots, such as ChatGPT, Surameery & Shakor (2023), Amaro et al. (2023), and Ross et al. (2023) stated that these tools were not without flaws (Surameery & Shakor, 2023), occasionally unreliable (Amaro et al., 2023), the answers were not always correct (Ross et al., 2023), and the results generated by these tools should be interpreted and verified by a human (Amaro et al., 2023). Nevertheless, Amaro et al. (2023) and Ross et al. (2023) reported the overall user experience with LLM chatbots to have been positive. According to Hollensen (2014, pp. 153, 409-413), positive or negative customer experience is a result of meeting or not meeting customer expectations. Since the user experiences with LLM chatbots were found to be positive, even though the tools occasionally provided inaccurate responses, it could be said that Partridge's (1998) claim regarding AI tools causing disappointment in the users is not necessarily true in the contemporary environment of AI tools.

4.1.4 Risks

Risk refers to a potential of there being negative outcomes from the use of AI tools (Cambridge, 2024), for example, compromise of organizational assets via unauthorized use (Talbot & Jakeman, 2009). To manage risk, one can accept a risk to pursue an opportunity, or retain the risk by an informed decision (ISO, 2009). This subcategory regarding risks discussed topics such as careless use, deliberate misuse, lack of legislation and regulation, security concerns, inaccuracy, lack of transparency, and ethical concerns. The findings of this subcategory were compiled from 6 different critical findings, which are summarized below.

AI tools can be seen as power tools, and as such, when used responsibly, can be very beneficial, but if used carelessly, can cause significant risks (Partridge, 1998, p. 251). This point of view by Partridge is from the year 1998 and does not necessarily accurately describe the state of AI tools in the year 2024. Using public GPT chatbots for assistance with company's software development related activities is one such careless way of using AI tools. For data, which is to be held private and secure, public GPT's cannot be used, unless there is licensing with the service, which guarantees security and privacy of the information. It is possible to get answers by proxy from such GPT chatbots by encrypting the prompts into a form that the service cannot use for any other purposes. To do this, the person encrypting the prompts should be well aware of what he is doing, so he does not accidentally leak confidential information.

ChatGPT's moral issues include data bias, leaking of private information, breach of confidential information (Bale et al., 2023), plagiarism, bias, and lack of transparency (Fraivan & Khasawneh, 2023). ChatGPT's functionality requires attention in quality assurance, data protection, and security issues (Gill & Kaur, 2023). Exploring areas such as transfer learning, explainability, as well as detecting and mitigating bias is warranted to advance the prudent use of ChatGPT (Fraivan & Khasawneh, 2023). There are conflicted views on whether the use of ChatGPT should be limited or regulated. ChatGPT presents numerous challenges regarding practicalities, ethics, morals, and policy. In cases of deliberate misuse, regulators cannot punish perpetrators. Therefore, to manage GPT-tools, it is critical to create new legislation. (Dwivedi et al., 2023).

Regarding ethical concerns, data mining open-source repositories to obtain training data for creating code-generating AI tools can break licensing rules of the codes within the repositories. When using open-source

code, it is often required to give credit or attribution to the creator of the code. It is unlikely that AI tools will follow even simple licensing rules when generating code, while having part of their training data originate from open-source repositories, which is both an ethical and a legal issue.

Security issues, such as breach of confidential information and data protection, can be circumvented by two routes. The first route is to have a privately owned AI tool locked into the company's intranet. This way, the information the AI tool receives will not leak outside the company. The second route is to use an AI service with licensing, which guarantees secure and confidential handling of information it receives. Which route the company chooses to use depends on a cost-risk assessment. In terms of costs, is it cheaper to use an outsourced service? In terms of security, does the company think an outsourced service is secure and trustworthy enough?

Using a company-owned service has more options regarding customizability. The training data can be customized to fit the company's specific needs. Also, a privately owned AI tool will have advantages regarding transparency and bias since the company has access to the AI tool's codebase and can diagnose the source of its behavior and even improve its functionality. Also, some data filtering features of the AI tools can be modified to better suit the company's needs. On the other hand, since this is a privately created AI tool, there could be some mistakes done in the process of creating this tool, which could create another set of security issues, such as the company's employees coming up with ways to bypass the security parameters of the AI tool, and by doing so, possibly gaining access to confidential information. This could be mitigated by creating strict contractual policies with the employees, as well as by having all the interactions with the company tool strictly monitored. In addition, there could be functionalities written into the AI tool which will create an alarm in cases where it seems like someone is using a technique for circumventing the AI tool's security filters.

4.2 Requirement Engineering and System Design

Requirements engineering is used for finding out the demands and wishes from the stakeholder and cultivating them into a collection of requirements, which function as a foundation for the subsequent

development tasks (Zhi, 2018). In *system design*, the software architects utilize system requirements for creating visualizations regarding the application's user interface, behavior, and underlying architecture (Github, 2023). The compiled findings from the articles found AI tools to have applications in both requirement engineering and system design, and there were demonstrated techniques for successfully automating tasks in both requirement engineering and system design, such as a requirements elicitation chatbot and user interface generation. There were 8 positive findings classified into this section, which are summarized below.

ML has applications for requirement engineering, such as requirements modeling and analysis, requirements selection, requirements prioritization, requirements classification, and requirements tracing (Shafiq et al., 2021). Panichella & Ruiz (2020) demonstrated a text classification tool capable of effectively classifying specifications and user feedback. Surana et al. (2019) presented an automated requirements elicitation process using an AI chatbot.

It seems automating the process of gathering requirements and feedback from users is rather feasible to implement using current technologies (Surana et al., 2019) (Panichella & Ruiz, 2020). It would be useful to conduct research investigating how accommodating it is for the client(s) to be giving system requirements related information via a chatbot system. Furthermore, it would be interesting to hear numbers regarding how successful such initiatives are in real life projects, and what are the setbacks of using such systems. In a situation where a human is conducting requirements elicitations, the human can ask follow-up questions from the interviewees regarding issues which remain unclear.

In software development projects, communication between software developers and customer(s) is already difficult, which leads to some projects gathering inaccurate requirements, leading to the need for modification of the software product afterwards. Automating this crucial process can cause further issues in this area. However, combining automated requirements elicitation with human efforts can bring benefits to the requirements elicitation process. Since the mundane requirements get gathered efficiently using automation, this frees up the hands of the human labor to focus on more complicated requirements elicitation tasks, increasing the quality of the work.

For architecture and design, ML can be applied in the following tasks: design and modeling; design pattern prediction; development effort estimation (Shafiq et al., 2021); and user interface generation

(Fraïwan & Khasawneh, 2023). Bale et al. (2023) advocates leveraging ChatGPT for streamlining user interface and user experience interactions.

Automated system redesign and reconfiguration can be used to perform a beneficial role: as a software project nears its completion, its original requirements often change, thus a need arises for design via experimentation, which is feasible to implement using automated code generation. Many relevant automated code generation techniques have been demonstrated, thus design via experimentation is a viable option to consider and develop further. Performing systematic changes within a system can be efficiently and controllably performed using a program manager, a program which can be automated. (Ebbah, 2002).

4.3 Code Writing

In coding phase, programmers turn the specifications gathered from the client into code, converting the specifications into a software product (Github, 2023). The findings of this section, consisting of 7 positive findings and 1 critical finding, found AI techniques to have numerous applications for the implementation phase, such as code generation, prototype generation, and a coding assistant chatbot.

According to Shafiq et al. (2021), ML has the following applications in the implementation phase: code cloning, code localization, code refactoring, and code labeling. LLMs trained using big code have applications in various code writing related tasks, such as generating code, offering code completion suggestions, translating code to other languages, refining code, detecting clones, and detecting defects (Man-Fai et al., 2023).

ChatGPT can be used for generating code, increasing reliability and dependability of code, and optimizing algorithms (Gill & Kaur, 2023). ChatGPT has performed well in code generation tasks. Even though ChatGPT can produce code, it does not comprehend dynamic code behavior, meaning how the code behaves once the program is running, but it grasps syntax rules, and to an extent, static code behavior. (Ma et al., 2023). ChatGPT can be used for helping in code writing, but because it cannot comprehend dynamic code

behavior, it can't reliably write properly functioning code. Therefore, it is left to the developer to evaluate and test the code ChatGPT has offered.

ChatGPT could excel in tasks relating to finding small syntax errors, such as missing brackets. Small syntax errors can cause cascading errors, meaning one small syntax error can cause numerous other errors, which might make the programmer think there are a lot of errors in the code. With the help of ChatGPT, it is possible to do a routine check for syntax correctness to prevent such cascading errors. Another potential application for ChatGPT is to give the compiler errors for ChatGPT to analyze and locate the cause of the error(s).

An LLM-based chatbot prototype, '*coder's assistant*', was able to provide high quality assistance and increase the productivity of its users by producing and explaining code, in addition to helping with questions related to both programming and general knowledge. The answers the coder's assistant was able to provide were not always correct, nonetheless, the overall user experience with the assistant was positive. (Ross et al., 2023.)

According to Raza (2009), it is possible to automate the software development's coding phase by using genetic code. By using genetic algorithms, it is possible to automatically generate computer programs (Raza, 2009). Some techniques which have been demonstrated in automated development tools include: language feature, which creates quick prototypes, leading to efficient and easily modifiable codes; meta programming, which uses parser generators and interpreters to create runnable code; program browsers, which helps the programmer look at the code and give smart recommendations; and automated data structuring, which automatically turns a data structure specification to an implementation structure (Ebbah, 2002).

4.4 Testing, Quality Assurance, and Software Analysis

Testing is done before software is published to production, ensuring it is safe, reliable, and free from bugs and errors (Github, 2023). *Quality assurance's* goal in context of software development is to make sure the product meets quality requirements (ISO, 2015), including functional requirements. *Software analysis* is

done using methods and tools for diagnosing and testing the software to better meet specifications and enhance software quality, leading to increased reliability, performance, and security (Thadt, 2020). There were 7 positive and 3 critical findings classified into this section, which are summarized below.

ML has applications in software testing in the following areas: generating test cases; generating test data; test oracle generation, which evaluates the outputs of test runs (Hossain et al., 2023); and testing effort estimation (Shafiq et al., 2021). Applying current NLP-techniques in testing lowers costs in designing test-cases and saves manpower for testing related tasks. Using NLP-techniques for generating tests scored between 70% and 90% for accuracy, which is effective for producing suitable test cases, but for industry wide use, the accuracy needs to be close to 100%. (Garouis et al., 2020).

ChatGPT cannot understand dynamic code (Ma et al., 2023), and therefore it cannot be used for testing purposes in the context of determining if code does what it is supposed to do once it is running. Nevertheless, ChatGPT has functionality, which is relevant to testing in other ways, such as giving analytical advice, or helping locate sources of the problem in cases where testing has found an issue. It is likely that other AI tools besides ChatGPT have this same lack of being incapable of understanding dynamic code behavior. Therefore, when introducing an AI tool for testing applications, creating code, or evaluating code, its capacity for understanding dynamic code should be tested first.

Automated software testing empowers present-day software development, which benefits from quick feedback on the quality of the product. Implementing script-based test automation is challenging, and the possibility of failure is significant. (Wiklund et al. 2017.)

Wiklund et al. (2017) states it is common for projects facing tight schedules to deviate from good testing practices. Using AI tools, it is possible to quickly generate code that seemingly works and meets requirements, but if the project is under time pressure, the verification and testing of the product can be done insufficiently. Since both reliability and security are key issues with AI generated code, it is especially vital to make sure it is thoroughly tested and verified to meet the requirements. The privilege of having access to AI tools comes with the responsibility of using them with high regard.

According to Shafiq et al. (2021), ML was found to have the following applications in quality assurance: fault prediction; bug prediction; and defect prediction (Shafiq et al., 2021). ChatGPT can be used for increasing reliability and dependability of code (Gill & Kaur, 2023). ML can be used for software analysis,

technique assessment, software process assessment, as well as verification and validation (Shafiq et al., 2021).

To summarize this section, AI tools were found to have useful roles in tasks surrounding testing, such as creating test cases and test data. There were no findings indicating that AI tools can be used for directly running test cases; however, this task can be automated using scripts (Wiklund et al., 2017). The findings indicated AI techniques to have applications in both quality assurance and software analysis.

4.5 Debugging and Maintenance

Debugging is the process used when software is not working as specified, and programmers find and fix errors by studying the code: the code is checked thoroughly, analyzed, and a solution is applied (Amazon Web Services, 2024). *Software maintenance* is a process of ensuring a product's capacity to deliver a service by fixing or preventing issues, and enhancing performance or features (ISO, 2022). AI tools were found to have applications in software maintenance and be well-suited for debugging and reverse engineering. There were 8 positive findings and 1 critical finding classified into this section, which are summarized below.

For software maintenance, ML has applications in the following tasks: predicting software maintainability; detecting aging software; and estimating maintenance effort (Shafiq et al., 2021). LLMs trained with big code have applications in debugging related tasks like code refinement, defect detection, and clone detection (Man-Fai et al., 2023). Ahmed et al. (2021) demonstrated an automated bug reporting tool which scored accuracies as high as 88% for category prediction and 90% accuracy for priority prediction.

ChatGPT can be used for debugging and optimizing algorithms (Gill & Kaur, 2023). ChatGPT is well suited for debugging, predicting bugs, and explaining bugs, but it is not without flaws, and best results can be obtained by using ChatGPT in unison with other debugging techniques and tools (Surameery & Shakor, 2023).

Reverse engineering is used to increase our understanding of the code, which in turn makes it possible to recognize vulnerabilities in the code. Using probabilistic models, which utilize big code to support analysis

of the code, can assist the reverse engineering process by automatically recovering information which is strenuous and difficult to locate manually, such as targets of calls or types of objects in binaries. Probabilistic models can be utilized for locating code pieces which are similar to a vulnerable piece of code, preventing future problems by correcting all vulnerable parts of the codebase. (Yahav, 2015.)

AI tools are especially apt for noticing small details in a large set of data, which is an important feature for debugging: sometimes a seemingly large problem can arise from a small mistake due to human negligence. An AI tool has the potential to quickly notice such small mistakes, which will save a lot of effort on the part of the human debugger.

4.6 Documentation

Software development *documentation* is produced for tracking software's development and use from beginning to end. Documentation supports developers in developing software, and helps users in understanding, operating, and troubleshooting the software product. (Indeed, 2023). AI-tools were found to be well-suited for documentation tasks. There were 3 positive findings classified into this section, which are summarized below.

ChatGPT can be used to help with documentation by using it to describe code (Gill & Kaur, 2023). According to Ma et al. (2023), ChatGPT excels in tasks related to document generation. LLMs have applications in code summarization (Man-Fai et al., 2023).

GPT-tools are especially useful for creating documentation, since they excel in taking in large amounts of information, and then summarizing the data. A problem relating to giving a GPT-tool this information for analysis is security related: there needs to be a guarantee that the information given to the GPT-tool won't be misused later. This can be achieved by using a model which has been created and trained by the company itself, which is possible, since there is publicly available source code for creating GPT-related technologies.

It was surprising there were only a few findings in this category regarding documentation, since AI tools excel in document generation tasks (Ma et al., 2023). The reason for the lack of findings in this category

could be the idea that post-documentation is not seen as a challenging software development issue since post-documentation is rarely a bottleneck for the completion of a project. However, writing pre-documentation is not an easy task, and good pre-documentation supports in the creation of good code (Dijkstra, 1967-1970/2007). It would be useful to utilize AI tools for helping generate pre-documentation, and this generated pre-documentation could then be utilized in the creation of code, as well as supporting in the verification of code.

4.7 Software Engineering Research

Shafiq et al. (2021) found the research evaluating ML techniques in software engineering is empirical and evidence based. According to Bale et al. (2023), ChatGPT can be used for advancing software engineering research. Examining repositories of big code and recognizing natural patterns can lead to more efficient tools for AI-assisted software development (Man-Fai et al., 2023). In the field of LLMs, there are substantial research opportunities, such as developing LLMs which prioritize privacy, fairness, interpretability, and transparency, leading to more responsible, user-oriented, and versatile AI programming assistance applications, which will reduce the barriers of use for AI tools, as well as lead to more capable programming processes (Man-Fai et al., 2023). Further research into ML, NLP, and deep learning can address fundamental issues in the field of software testing (Boukhilif et al., 2023).

There were 4 positive findings classified into this section dealing with software engineering research. In summary, software engineering research was found to benefit AI tools; also, AI tools were found to be beneficial to software engineering research; and the research regarding ML techniques was found to be empirical.

5 DISCUSSION

5.1 Aims & Objectives

The research objective in the literature synthesis and analysis was to evaluate the applicability of automation and AI in software development, and what consequences entail from utilizing automation and AI in software development. The results helped identify advantages and disadvantages of AI and automation in software development, and what SDLC processes it is possible to automate using current technologies. The threats, solutions to threats, and opportunities connected to utilizing automation and AI in software development are analyzed in the SWOT analysis subsection of this Discussion chapter.

The research objectives of the bibliometric analysis were to identify themes, trends, and research contributors. The results were able to recognize article publication and citation patterns; country collaboration patterns; popular topics, keywords, and abstract words; as well as keyword clusters and patterns in these clusters.

5.2 Data Collection

Both research phases, bibliometric analysis and selective literature synthesis and analysis, utilized thematic analysis as an analysis method and followed PRISMA guidelines in data collection. In the bibliometric analysis, a search was conducted in Scopus database, yielding 1049 conference papers and journal articles, which were refined using exclusion criteria down to 480 articles. For the selective synthesis and analysis, 6 articles were chosen from the 480 articles used in the bibliometric analysis. From the references of these 6 articles, using inclusion criteria, additional 24 articles were selected, resulting in 30 articles in total, which were used for conducting a literature synthesis and analysis.

5.3 Bibliometric Analysis

5.3.1 Analysis of Productivity in the Dataset

The numbers of yearly citations and yearly productions closely follow each other, apart from the years 2022 and 2023, since these years have not had time to accumulate citations. The best years in terms of production numbers and citations were the years from 2012 to 2014, while years from 2018 to 2021 also had good numbers in terms of production and citations. Most cited papers were from the years 2012, 2013, 2014, and 2018.

In the period between 2010 and 2014, countries of United States, Canada, Germany, India, and China were the highest producers of articles. In the 2015 to 2023 period, countries of India, China, Pakistan, Malaysia, Indonesia, and Saudi Arabia raised their article production numbers, with India increasing its production 4-fold, while United States and Canada reduced their production to one third of their 2010 to 2014 levels, and Germany maintained its previous level of production.

Canada and India were the most cited countries by a significant margin. Belgium, Portugal, Canada, and Norway had the highest average article citations. European countries had a significant presence in both highest average article citations and total citations, while Germany was the only European country who was in the list of ten countries with the highest productions of articles.

Regarding country collaboration patterns, the following trends were identified: North American and Asia Pacific countries collaborated with each other; Middle Eastern countries collaborated; and European and Southeast Asian countries had collaboration. United States, Saudi Arabia, and Italy had the richest collaboration networks with other countries.

Canadian, European, Indian, and Pakistani authors were the most prolific publishers of articles, whose articles focused on themes of testing, automation, and different AI techniques. The most cited authors were from the countries of United States, Singapore, Portugal, Belgium, and Canada, who discussed topics of AI and automation in the contexts of testing, debugging, and code writing. The most cited papers in the

dataset discussed AI and automation in debugging, testing, quality assurance, and requirements engineering.

5.3.2 Analysis of Dataset Content

Keywords used by authors in the dataset articles mostly comprised of AI related techniques, such as 'machine learning', 'natural language processing', 'artificial intelligence', and 'deep learning.' 'Machine learning' was the most used keyword by a significant margin. Keywords 'automation' and 'testing' were also common occurrences. Keywords 'machine learning', 'automation', and 'natural language processing' started to be used in the dataset since the year 2012. The Keyword 'artificial intelligence' started to be used in the dataset since 2013, and the keyword 'deep learning' since 2019. Words found from abstracts relating to AI included 'machine learning', 'defect prediction', 'support vector machines', 'software fault prediction', and 'natural language processing.'

Author's keywords were algorithmically formed into four clusters. The keyword 'machine learning' was clustered with keywords connected to giving predictions, recommendations, and classifications. The cluster with keywords relating to testing and automation was linked with the keyword 'deep learning', which can be used for generating test cases. A cluster was formed with keywords 'automation' and 'DevOps', which refer to agile development and using DevOps techniques for automating and streamlining software development processes. The fourth cluster was centered around requirements engineering, which was associated with the keywords 'natural language processing' and 'artificial intelligence', both of which are central in automating requirements engineering processes.

5.4 Selective Literature Synthesis and Analysis

AI tools and algorithms were used in all SDLC phases and were found to be generalizable, adaptive, and scalable. AI tools are promising in terms of providing benefits to facilitating innovation, efficiency, and streamlining processes. Limitations of AI tools include inaccuracy, unreliability, as well as lack of

interpretability and explainability. These findings regarding limitations are based on articles discussing LLM-based chatbots, so they do not necessarily generalize to other AI tools. Risks connected to AI tools involved legal and ethical issues, security issues, and issues relating to quality assurance.

AI tools were found to be highly applicable for various requirements engineering and system design tasks, such as automated requirements elicitation (Surana et al., 2019) and user interface generation (Fraivan & Khasawneh, 2023). AI tools have a lot of applications in tasks relating to code writing, such as code generation and translating code to another language. In testing, AI tools were shown to have applications in test case generation, test data generation, test oracle generation, and test case design, among others.

Both code writing and testing had issues, which were addressed by Tufano et al.'s (2022) article, an article outside of the 30 articles analyzed. In code writing, there was doubt regarding the AI tool's understanding of dynamic code behavior, since it was a feature lacking in ChatGPT. In testing, there was no mention in the analyzed articles of using AI for directly running tests. Tufano et al. (2022) presented AutoDev, an AI-powered programming environment that autonomously performs development tasks, which can execute code and run tests (Tufano et al., 2024), showing there to be potential for an AI tool to perform these tasks. For quality assurance and software analysis, AI tools were found to be useful for predicting problems in the code, increasing reliability of the code, and verifying specifications are met.

AI techniques have a lot of applications in both debugging and maintenance, and ML excels in reverse engineering tasks, which are strenuous to perform manually (Yahav, 2015). For maintenance, AI tools can be used for detecting aging software, optimizing algorithms, and code refinement. For debugging, AI tools can be used for defect detection, explaining bugs, and clone detection.

For documentation, LLMs and ChatGPT were found to perform well in describing code, code summarization, and document generation. Software engineering research was found to be beneficial towards improving AI tools, and AI tools were found to be of assistance in software engineering research.

5.4.1 SWOT Analysis

Strengths

As was shown by the results, automation and AI technologies had applications in all SDLC phases and were found to be of assistance in all of them. Using AI tools offers ways to make software development more productive and efficient, creating cost and time reductions, and facilitating innovation. ML was found to be especially suited for poorly understood problems, domains with large datasets, and domains requiring adaptability to a changing environment (Zhang, 2000). ML algorithms can be embedded into software products, making them more autonomous and adaptive (Zhang, 2000).

Weaknesses

To incorporate automation technology into different SDLC processes, it is probable that a significant amount of effort and competence is required to be able to do this. The easiest and most convenient approach for incorporating AI related automation techniques is purchasing AI services from outside the company. Using outside of company AI tools can have problems regarding privacy, security, and leaking of confidential information. These tools might use the data they receive as training data for improving their efficiency. To avoid this issue with privacy and security, the company needs a good contract with the service provider, or the company needs to create a private AI tool. This need to have private AI tools could also create a market for creating and selling AI tools to companies for private use, such as providing GPT chatbots, which have been custom tailored to the customer's needs.

LLM-based chatbots were noted to occasionally produce unreliable and incorrect results (Ross et al., 2023). LLMs are liable to create nonfactual fabrications when analyzing code logic, showing there to be a need for verifying the materials produced by LLMs (Ma et al., 2023). LLMs' insufficient interpretability is a concern because of the rigorous risk management and reliability requirements involved in software engineering (Ma et al., 2023). In order to use LLMs responsibly, their outputs should be interpreted and verified by a human, and users should be made aware of the limitations chatbots have (Amaro et al., 2023).

Threats – job displacement

Classically, a perceived threat and fear regarding automation is job displacement effects. Automation and AI technologies are powerful tools, applicable to all phases of SDLC, and it is likely they will be utilized to their full effect with minimal restriction.

The current software developer tasks will be increasingly automated, which is likely to reduce the demand for software developers and change the way their work is done. It is also possible that instead of taking jobs from software developers, automation could free up the hands of software developers and allow them to move their work efforts from lower-level tasks into more higher-level tasks, increasing the quality of software products and giving the software company a competitive edge. Those developers who are the most skilled in utilizing the new AI technologies will thrive the most, being able to justify high wages and are sought after by companies.

Borjas (2012, p. 85) describes the function for labor demand as follows: $q = f(E, K)$, where q is the outputs of the company, E is the labor force employed, and K represents capital, such as land, machinery, or physical inputs. According to Borjas (2012, p. 84), companies employ labor (E) to satisfy the needs of the market for consuming products and services (q). In the software development industry, the demand for software products is quite vast, since the demand for software products is global, and there is demand for software products in almost all sectors of industry. In terms of raw materials (K), software development has no limitations since they are refining electronic bits into algorithms or software products.

In Finland, in the year 2020, there was a total of 53500 personnel working in the software development sector (Ek, 2020). In the year 2018, there was estimated to be 8000 unmet software developer jobs (Collin, 2019), and the number of unmet employee needs was estimated to have a growing trajectory (Mäntylä, 2017), showing there to be a significant lack of available workforce in the industry. According to a survey conducted to small and medium sized software companies, availability of workforce was found to be the biggest obstacle to growth (Ek, 2020).

In the years between 2013 and 2018, the software development industry's employment numbers in Finland grew on average about 6,5% per year (Ek, 2020). OECD estimated automation to displace 14% of jobs in 15-20 years, which is about 0,9% per year (OECD, 2019). Despite OECD having predictions of automation causing significant job displacement, there was found to be employment growth between the

years 2012 to 2019 across all observed 21 countries. Jobs which had a high risk of automation job displacement had lower employment growth (6%) compared to jobs with low risk of automation (18%), and the highest growth was found in the ICT sector (51%). (Georgieff & Milanez, 2021).

Applying OECD's 14% job displacement rate to the software development industry, it would mean that in 15 years Finland's 8000 missing workers from the industry would be filled by automation. Automation stepping in and filling the shoes of the missing workers is good news for the short term, helping software companies deliver products faster and supporting a higher level of quality. After this 15-year period, automation would start slowing the employment growth rate from 6,5% to 5,6% per year. This slowdown can be solved, if necessary, by training less software developers. This point of view is isolated from the possibility of AI technologies and the AI-industry creating new jobs.

Opportunities – Entrepreneurship

Entrepreneurship is a compelling opportunity in the emergence of AI technologies, which will be a new rising global market, offering numerous entrepreneurial opportunities. *Union Bank of Switzerland* (UBS) estimated the AI-industry to grow 15-fold from 2022 (\$28 billion) to 2027 (\$420 billion) (Fox, 2024), which is approximately 72% yearly growth. The company OpenAI has a net worth of \$86 billion (Levine, 2023) and, according to OpenAI COO Brad Lightcap, 1200 employees (Bloomberg, 2024). Translating this number into the UBS projection of 2027, we can estimate that in 2027, there will be 5800 jobs in the AI-industry.

The new and old forms of automation will help the software entrepreneur have efficient software development processes, further supporting the profitability of entrepreneurship in the contemporary environment. Considering automation will make software companies more efficient, as an employee of a software company, the employee's salary raises at a significantly slower rate than this new wave of AI technologies will raise the productivity of a software company. As an entrepreneur, the software developer will be able to see this increase in productivity directly affect the earnings he receives as an entrepreneur, while as an employee, this would not be the case.

UBS recommends investors to invest in software companies to increase their portfolio in the AI-trade, which the UBS sees as the tech theme of the decade (Fox, 2024), showing there is potentially ample

investment capital available for software companies. However, bank loans are difficult for software companies to secure, because software companies differ from typical companies, not necessarily having large balances of equipment or capital on their accounts, and the company's value lies in its expertise (Ek, 2020). Software companies have only a few expenses, the major expenses being salaries of employees and workspace rent. Starting a software company does not require a large starting capital (Ek, 2020), only enough capital to cover the company expenses until the business starts to generate enough profit to cover its own running expenses. Entrepreneurship will help the entrepreneur himself by generating earnings; other software developers by offering jobs; and society at large by employing workers and providing tax revenues.

AI algorithms can be added into software products, providing improvements to what they were added into, giving AI algorithms a vast array of contexts they can be applied to, creating a new paradigm for gaining competitive advantage. Assuming arbitrarily that adding one AI algorithm to the business processes or software products gives the company 1% improvement in terms of increasing efficiency, raising quality, or streamlining processes. If you repeat this process 70 times ($101\%^{70} = 200\%$), it will add up to doubling the efficiency of the company, resulting in a significant competitive advantage. As an example of benefits yielded using an AI tool, a study by Peng et al. (2023) found programmers completing their programming tasks 55% faster when paired with GitHub CoPilot. Software developers and software companies who learn to utilize AI algorithms and AI tools most rapidly and to the greatest extent will be able to have a dominant position in the market.

Considering there is a new wave of AI technologies coming to both software development industry and all industries at large, there will be a lot of business ideas for finding ways to incorporate these AI technologies into existing industries. There will be a need for skills relating to machine intelligence and applying AI tools for process automation and process streamlining. The job of the software developer will be to bring these technologies to use and apply them in different contexts, facilitating the adoption of AI tools for increasing productivity and efficiency of different industries. Because of this new wave of AI technologies and the need for incorporating these technologies into old models, there will likely be a lot of jobs and opportunities for software developers. The old software developer jobs might be partly displaced by IA tools, but there will be new jobs created for software developers with these new technologies.

5.5 Limitations

The findings of this paper are limited to the topics and articles used in the two research methods, bibliometric analysis and selective literature synthesis and analysis. The data produced by the Biblioshiny application in the bibliometric analysis is not without flaws. Some of these flaws were noticed and corrected during the research process, while some flaws may have gone undetected. The citation numbers close to the year 2024 have not had time to accumulate citations, so the results regarding the citation numbers from years 2023 and 2022 do not necessarily represent reality.

In the selective literature synthesis and analysis, the chosen articles could have contained additional findings which may have been omitted due to the high complexity of the articles and limited time resources. Limitations in time resources also prevented investigation of a larger pool of articles, which would have provided more comprehensive results.

5.6 Validity and Reliability

Regarding the bibliometric analysis, if someone were to use the same methods used in this paper, would they receive the same results? The results produced by the Biblioshiny application would look similar, but over time, the citation counts of the articles would change, making the citation counts for the years 2022 and 2023 more accurate. The interpretations done on the results might differ, since this is qualitative in nature, but the additional interpretations would supplement the interpretations offered by this paper, giving us a better understanding of the research trends in this field.

Regarding the selective literature synthesis and thematic analysis part of this paper, it is important to note that the results of a qualitative research are not universally applicable, and the conditions cannot necessarily be recreated (Castleberry & Nolen, 2018). If someone were to use the same articles for conducting thematic analysis as was done in this paper, they probably would be able to gather the same key findings from the articles as this paper has, and perhaps be able to find additional findings or able to perceive the findings in a more accurate manner. Regarding the interpretations of the findings, they could slightly differ, but the interpretations would complement the views offered by this paper. Since the results

are qualitative in nature, the readers can evaluate whether the conclusions relate to their field of speciality (Castleberry & Nolen, 2018).

5.7 Conclusion

The bibliometric analysis was responsible for answering RQ1: “What is the current state of research on AI and automated software development?” There were no previous bibliometric analyses on the topic of ‘automation and AI in software development’, and the bibliometric analysis conducted in this paper will meet this research gap and contribute to the research field. The conducted bibliometric analysis identified a growing trend in publication numbers, indicating a growing research activity. Citation numbers showed a mixed pattern with two peaks, the earlier peak being dominant, showing the earlier research having an enduring impact. The most cited articles in the dataset discussed topics of automated testing, debugging, and code writing, as well as using machine intelligence to perform predictions such as test case prioritization, requirements prioritization, and bug prediction. Recognizing these key topics provides an insight into the current research interests and emerging trends in the field.

The selective literature synthesis and analysis was conducted to answer research questions RQ2: “What SDLC phases and processes are AI and automation applicable to?”, RQ3: “What are the advantages and disadvantages of applying different AI and automation techniques to SDLC processes?”, and RQ4: “What are the problems, solutions to problems, and opportunities presented by applying automation and AI to SDLC processes?” A research gap was identified in the topic of utilizing AI in different SDLC phases due to the topic of AI in software development being extensive and too broad (Barenkamp et al., 2020). The answers to RQ2, RQ3, and RQ4 will assist in furthering discussion and understanding in the field, offering a basis for upcoming research.

To answer RQ2, automation and AI were found to have applications in all phases of SDLC and a great deal of SDLC processes, which implies that over time, there will be a greater adoption of automation and AI into SDLC processes. Addressing RQ3, AI related automation was found to be highly generalizable, adaptive, and scalable, in addition to increasing the efficiency of processes, while script-based automation can run direct tests, which are difficult for an AI tool to perform. Disadvantages of the use of AI tools included

privacy and security concerns, as well as difficulties with implementation. All these issues can be solved by practical measures. These findings encourage further deployment of AI and automation into SDLC processes.

RQ4 was answered by investigating the threat of automation causing job displacement in software development, which was not found to be a concern in the 15-year range of analysis, after which the job displacement effects can be solved by slightly reducing the training of new software developers. The discovered opportunities included the emergence of AI technologies creating jobs, providing entrepreneurial opportunities, and enhancing the profitability of software companies by increasing the efficiency of SDLC processes.

5.8 Future Work

The findings indicated ways we can improve our automated code generation capabilities, as well as the reliability and understanding of this generated code. Firstly, investigating use of AI tools for analyzing AI generated code offers ways for improving the reliability of code. Secondly, using pre-documentation to generate code with an AI tool is a research topic which could yield interesting results and ways to improve the quality and reliability of AI generated code. Thirdly, finding ways to generate pre-documentation using an AI tool could help software developers improve their understanding of coding problems, assist with code writing, as well as work as a basis for generating code with an AI tool.

REFERENCES

- Ahmed, H. A.; Bawany, N. Z.; & Shamsi, J. A. (2021). CaPBug-A Framework for Automatic Bug Categorization and Prioritization Using NLP and Machine Learning Algorithms. *IEEE Access, Volume 9, Pages 50496-50512*. DOI: <https://doi.org/10.1109/ACCESS.2021.3069248>
- Amaro, I.; Greca, A. D.; Francese, R.; Tortora, G.; & Tucci, C. (2023). AI Unreliable Answers: A Case Study on ChatGPT. *4th International Conference, AI-HCI 2023 Held as Part of the 25th HCI International Conference, HCII 2023, Copenhagen, Denmark, July 23–28, Proceedings, Part II*. DOI: https://doi.org/10.1007/978-3-031-35894-4_2
- Amazon Web Services. (2024). What is Debugging? URL: <https://aws.amazon.com/what-is/debugging/> (Retrieved: 21.2.2024)
- Amazon Web Services. (2024). What is GPT? URL: <https://aws.amazon.com/what-is/gpt/> (Retrieved: 5.5.2024)
- Bale, A. S.; Vada, Y. R.; Oshiojum, B. E.; Lakkineni, U. K.; Rao, C.; Venkatesh, K.; & Rani, I. (2023, July 12). ChatGPT in Software Development: Methods and Cross-Domain Applications. *International Journal of Intelligent Systems and Applications in Engineering, 2023, Volume 11, No. 9s, Pages 636-643*.
- Barenkamp, M.; Rebstadt, J.; & Thomas, O. (2020). Applications of AI in classical software engineering. *AI Perspect 2, Article 1*. DOI: <https://doi.org/10.1186/s42467-020-00005-4>
- Bloomberg. (2024, April 4). OpenAI COO: Working on Publishing Partnerships. *Youtube*. URL: <https://www.youtube.com/watch?v=CuAOi1L8SPY> (Retrieved: 13.4.2024)
- Borjas, George J. (2012, May 16). Labor Economics (6th Ed.). *McGraw-Hill, New York*.
- Boukhelif, M.; Hanine, M.; & Kharmoum, N. (2023). A Decade of Intelligent Software Testing Research: A Bibliometric Analysis. *Electronics, Volume 12, Issue 9*. DOI: <https://doi.org/10.3390/electronics12092109>
- Cambridge Dictionary. (2024). Adaptability. URL: <https://dictionary.cambridge.org/dictionary/english/adaptability> (Retrieved: 20.2.2024)

Cambridge Dictionary. (2024). Risk. URL: <https://dictionary.cambridge.org/dictionary/english/risk> (Retrieved: 20.2.2024)

Castleberry, A.; & Nolen, A. (2018, June). Thematic analysis of qualitative research data: Is it as easy as it sounds? *Currents in Pharmacy Teaching and Learning*, Volume 10, Issue 6, Pages 807-815. DOI: <https://doi.org/10.1016/j.cptl.2018.03.019>

Collin, Paula. (2019, March 4). Jopa 10 000 työpaikkaa koodareille, mutta tekijät puuttuvat. *Yle*. URL: <https://yle.fi/a/3-10669492> (Retrieved: 20.3.2024)

Denning, P. J.; & Lewis, T. G. (2019, November-December). Intelligence May Not Be Computable. *American Scientist*, Volume 107, Number 6, Page 346. DOI: <https://doi.org/10.1511/2019.107.6.346>

Di Stefano, J. S.; & Menzies, T. (2002). Machine learning for software engineering: case studies in software reuse. *14th IEEE International Conference on Tools with Artificial Intelligence, 2002. (ICTAI 2002). Proceedings., Washington, DC, USA, Pages 246-251*. DOI: <http://doi.org/10.1109/TAI.2002.1180811>

Dijkstra, E. W. (1967-1970). A Preliminary Investigation into Computer Assisted Programming (Original date 1967-1970, transcribed 2007 by Arnout Engelen). *The University of Texas*. URL: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD02xx/EWD237.html> (Retrieved: 22.3.2024)

Dwivedi, Y. K.; Kshetri, N.; Hughes, L.; Slade, E. L.; Jeyaraj, A.; Kar, A. K.; Baabdullah, A. M.; Koohang, A.; Raghavan, V.; Ahuja, M.; Albanna, H.; Albashrawi, M. A.; Al-Busaidi, A. S.; Balakrishnan, J.; Barlette, Y.; Basu, S.; Bose, I.; Brooks, L.; Buhalis, D.; Carter, L.; Chowdhury, S.; Crick, T.; Cunningham, S. W.; Davies, G. H.; Davison, R. M.; Dé, R.; Dennehy, D.; Duan, Y.; Dubey, R.; Dwivedi, R.; Edwards, J. S.; Flavián, C.; Gauld, R.; Grover, V.; Hu, M. C.; Janssen, M.; Jones, P.; Junglas, I.; Khorana, S.; Kraus, S.; Larsen, K. R.; Latreille, P.; Laumer, S.; Malik, F. T.; Mardani, A.; Mariani, M.; Mithas, S.; Mogaji, E.; Nord, J. H.; O'Connor, S.; Okumus, F.; Pagani, M.; Pandey, N.; Papagiannidis, S.; Pappas, I. O.; Pathak, N.; Pries-Heje, J.; Raman, R.; Rana, N. P.; Rehm, S-V.; Ribeiro-Navarrete, S.; Richter, A.; Rowe, F.; Sarker, S.; Stahl, B. C.; Tiwari, M. K.; van der Aalst, W.; Venkatesh, V.; Viglia, G.; Wade, M.; Walton, P.; Wirtz, J.; & Wright, R. (2023). Opinion Paper: “So what if ChatGPT wrote it?” Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy. *International Journal of Information Management*, Volume 71. DOI: <https://doi.org/10.1016/j.ijinfomgt.2023.102642>

- Ebbah, J. O. G. (2002). Deploying Artificial Intelligence Techniques in Software Engineering. *American Journal of Undergraduate Research*, Volume 1, Issue 1. DOI: <http://doi.org/10.33697/ajur.2002.006>
- Ek, Johanna. (2020). Ohjelmistoalan toimialaraportti 2020. *Työ- ja elinkeinoministeriö, Helsinki*. URL: <https://urn.fi/URN:ISBN:978-952-327-493-8> (Retrieved: 19.3.2024)
- Findley, M. G.; Kikuta, K.; & Denly, M. (2021). External Validity. *Annual Review of Political Science*, Volume 24, Pages 365-393. DOI: <https://doi.org/10.1146/annurev-polisci-041719-102556>
- Fox, Matthew. (2024). The growing adoption of AI will drive tech stocks higher for the rest of the decade, UBS says. *Business insider*. URL: <https://markets.businessinsider.com/news/stocks/tech-stocks-outlook-generative-ai-adoption-decade-artificial-intelligence-companies-2024-1> (Retrieved: 19.3.2024)
- Fraiwan, M.; & Khasawneh, N. (2023). A Review of ChatGPT Applications in Education, Marketing, Software Engineering, and Healthcare: Benefits, Drawbacks, and Research Directions. *arXiv - Computer Science - Computers and Society*. DOI: <https://doi.org/10.48550/arXiv.2305.00237>
- Garousi, V.; Bauer, S.; & Felderer, M. (2020). NLP-assisted software testing: A systematic mapping of the literature. *Information and Software Technology*, Volume 126. DOI: <https://doi.org/10.1016/j.infsof.2020.106321>
- Garousi, V.; & Mäntylä, M. V. (2016). When and what to automate in software testing? A multi-vocal literature review. *Information and Software Technology*, Volume 76, Pages 92-117. DOI: <https://doi.org/10.1016/j.infsof.2016.04.015>
- Georgieff, A. & Milanez, A. (2021). "What happened to jobs at high risk of automation?", OECD Social, Employment and Migration Working Papers, No. 255. *OECD Publishing, Paris*. DOI: <https://doi.org/10.1787/10bc97f4-en>. (Retrieved: 20.3.2024)
- Gill, S. S.; & Kaur, R. (2023). ChatGPT: Vision and challenges. *Internet of Things and Cyber-Physical Systems*, Volume 3, Pages 262-271. DOI: <https://doi.org/10.1016/j.iotcps.2023.05.004>
- GitHub. (2023). Understanding the SDLC: Software Development Lifecycle Explained. URL: <https://resources.github.com/software-development/what-is-sdlc/> (Retrieved: 20.2.2024)

- Goodfellow, Ian; Bengio, Yoshua; & Courville, Aaron. (2016). Deep Learning. *MIT Press, Massachusetts*.
URL: <http://www.deeplearningbook.org>
- Hayes, A. (2023, July 28). Scalability: What a Scalable Company is, and Examples. *Investopedia*. URL: <https://www.investopedia.com/terms/s/scalability.asp> (Retrieved: 20.2.2024)
- Hollensen, Svend. (2019, January 14). Marketing Management: A Relationship Approach (4th Ed.) *Pearson Benelux, Amsterdam*.
- Hossain, S. B.; Filieri, A.; Dwyer, M. B.; Elbaum, S.; & Visser, W. (2023, November). Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned. *ESEC/FSE 2023 - Proceedings of the 31st ACM Joint Meeting European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Pages 120-132*. DOI: <https://doi.org/10.1145/3611643.3616265>
- IBM. (2024). What are large language models (LLMs)? URL: <https://www.ibm.com/topics/large-language-models> (Retrieved: 5.5.2024)
- IBM. (2024). What is deep learning? URL: <https://www.ibm.com/topics/deep-learning> (Retrieved 5.5.2024)
- IBM. (2024). What is machine learning (ML)? URL: <https://www.ibm.com/topics/machine-learning> (Retrieved: 5.5.2024)
- IBM. (2024). What is natural language processing (NLP)? URL: <https://www.ibm.com/topics/natural-language-processing> (Retrieved: 5.5.2024)
- Indeed. (2023, March 11). 14 Types of Software Development Documentation. *Indeed*. URL: <https://www.indeed.com/career-advice/career-development/documentation-for-software-development> (Retrieved: 21.2.2024)
- ISO. (2009). ISO 31000:2009, Risk management - Principles and guidelines. *International Organization for Standardization*.
- ISO. (2015). ISO 9000, Quality management systems - Fundamentals and vocabulary. *International Organization for Standardization*.

ISO. (2022). ISO/IEC/IEEE 14764:2022(E), Software engineering – Software life cycle processes – Maintenance. *International Organization for Standardization*.

Iyer, S.; Konstas, I.; Cheung, A.; & Zettlemoyer, L. (2018). Mapping Language to Code in Programmatic Context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1643–1652, Brussels, Belgium. Association for Computational Linguistics. DOI: <https://doi.org/10.18653/v1/D18-1192>

Jayavel, S.; Arumugam, S.; Singh, B.; Pandey, P.; Giri, A.; & Sharma, A. (2013, November 30th). Use of Artificial Intelligence in Automation of Sequential Steps of Software Development / Production. *Journal of Theoretical and Applied Information Technology*, Volume 57, Issue 3.

Khomh, F.; Adams, J.; Fokaefs, M.; & Anoniol, G. (2018). Software Engineering for Machine-Learning Applications: The Road Ahead. *IEEE Software*, Volume 35, Issue 5, Pages 81-84. DOI: <https://doi.org/10.1109/MS.2018.3571224>

Levine, Matt. (2023, November 27). OpenAI Is Still an \$86 Billion Nonprofit. *Bloomberg*. URL: <https://www.bloomberg.com/opinion/articles/2023-11-27/openai-is-still-an-86-billion-nonprofit> (Retrieved: 13.4.2024)

Ma, W.; liu, S.; Wang, W.; Hu, Q.; Liu, Ye; Zhang, C.; Nie, L.; & Liu, Yang. (2023). The Scope of ChatGPT in Software Engineering: A Thorough Investigation. *arXiv – Computer Science - Software Engineering*. DOI: <https://doi.org/10.48550/arXiv.2305.12138>

Man-Fai, W.; Shangxin, G.; Ching-Nam, H.; Siu-Wai, H.; & Chee-Wei, Tan. (2023). Natural Language Generation and Understanding of Big Code for AI-Assisted Programming: A Review. *Entropy*, Volume 25, Issue 6, Article Number 888. DOI: <https://doi.org/10.3390/e25060888>

Merriam-Webster Dictionary. (2024). Efficiency. URL: <https://www.merriam-webster.com/dictionary/efficiency> (Retrieved: 20.2.2024)

OECD. (2019). OECD Employment Outlook 2019: The Future of Work. *OECD Publishing, Paris*. DOI: <https://doi.org/10.1787/9ee00155-en>. (Retrieved: 19.3.2024)

Padmanaban, P. H.; & Sharma, Y. K. (2019, June). Implication of artificial intelligence in software development life cycle: a state of the art review. *International Journal of Recent Research Aspects, Volume 6, Issue 2, June 2019, Pages 93-928. ISSN: 2349-7688.*

Page M. J.; McKenzie J.E.; Bossuyt P.M.; Boutron I.; Hofmann T. C.; Mulrow C.D.; Shamseer L.; Tetzlaff J. M.; Akl E. A.; Brennan S. E.; Chou R.; Glanville J.; Grimshaw J. M.; Hróbjartsson A.; Lalu M. M.; Li T.; Loder E. W.; Mayo-Wilson E.; McDonald S.; Moher D. (2021). The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ* 372:n71. DOI: <https://doi.org/10.1136/BMJ.N71>

Panichella, S.; & Ruiz, M. (2020). Requirements-Collector: Automating Requirements Specification from Elicitation Sessions and User Feedback. *IEEE 28th International Requirements Engineering Conference (RE), Zurich, Switzerland, Pages 404-407. DOI: <https://doi.org/10.1109/RE48521.2020.00057>*

Partridge, Derek. (1998). Artificial Intelligence and Software Engineering, Understanding the Promise of the Future. *Glenlake Publishing Company Ltd., Chicago.*

Peng, S.; Kalliamvakou, E.; Cihon, P.; & Demirer, M. (2023, February 13). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv - Computer Science - Software Engineering. DOI: <https://doi.org/10.48550/arXiv.2302.06590>*

Raza, F. N. (2009). Artificial Intelligence Techniques in Software Engineering (AITSE). *Proceedings of the International Multi Conference of Engineers and Computer Scientists, Volume 1, March 18 - 20, Hong Kong.*

Rehan S. S.; Adams M.; Bandara W.; Leemans SJJ.; Ouyang C.; Hofstede A. H. M.; Weerd I.; Wynn M. T.; & Reijers H. A. (2020). Robotic Process Automation: Contemporary themes and challenges. *Computers in Industry, Volume 115. DOI: <https://doi.org/10.1016/j.compind.2019.103162>*

Rewini, H.; & Abd-El-Barr, M. (2005). Advanced Computer Architecture and Parallel Processing (page 66). *John Wiley & Sons. DOI: <https://doi.org/10.1002/0471478385>*

Ross, S. I.; Martinez, F.; Houde, S.; Muller, M.; & Weisz, J. D. (2023, March). The Programmer's Assistant: Conversational Interaction with a Large Language Model for Software Development. *Proceedings of the 28th International Conference on Intelligent User Interfaces, Pages 491–514. DOI: <https://doi.org/10.1145/3581641.3584037>*

Russell, Stuart J.; & Norvig, Peter. (2009, December 1). *Artificial Intelligence: A Modern Approach* (3rd ed.). Pearson, New Jersey.

Shafiq, S.; Mashkoor, A.; Mayr-Dorn C.; & Egyed, A. (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages. *IEEE Access*, Volume 9, Pages 140896-140920. DOI: <https://doi.org/10.1109/ACCESS.2021.3119746>

Shankar, S. P.; & Chaudhari, S. S. (2023). Framework for the Automation of SDLC Phases using Artificial Intelligence and Machine Learning Techniques. *International Journal on Recent and Innovation Trends in Computing and Communication*, Volume 11, Issue 6s, Pages 379–390. DOI: <https://doi.org/10.17762/ijritcc.v11i6s.6944>

Singh, S.; & Sambhav, S. (2023). Application of Artificial Intelligence in Software Development Life Cycle: A Systematic Mapping Study. *Micro-Electronics and Telecommunication Engineering, Lecture Notes in Networks and Systems*, Volume 617. Springer, Singapore. DOI: https://doi.org/10.1007/978-981-19-9512-5_60

Smith, J.; & Wei, R. (2024, March 19). Implications of Artificial Intelligence in the Software Development Life Cycle. *EasyChair Preprint no. 12604*.

Sofian, H.; Yunus, N. A. M.; & Ahmad, R. (2022). Systematic Mapping: Artificial Intelligence Techniques in Software Engineering. In *IEEE Access*, Volume 10, Pages 51021-51040. DOI: <https://doi.org/10.1109/ACCESS.2022.3174115>

Sorte, B. W.; Joshi, P. P.; & Jagtap, V. (2015). Use of artificial intelligence in software development life cycle—a state of the art review. *International Journal of Advanced Engineering and Global Technology*, Pages 398–403.

Surana, C. S. R. K.; Shriya; Gupta, D. B.; & Shankar, S. P. (2019). Intelligent Chatbot for Requirements Elicitation and Classification. *4th International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT)*, Bangalore, India, Pages 866-870. DOI: <https://doi.org/10.1109/RTEICT46194.2019.9016907>

- Surameery, N. M. S.; & Shakor, M. Y. (2023). Use Chat GPT to Solve Programming Bugs. *International Journal of Information technology and Computer Engineering Volume 03, Issue 01, Pages 17-22, Dec 2022 - Jan 2023*. DOI: <https://doi.org/10.55529/ijitc.31.17.22>
- Talbot, J.; & Jakeman, M. (2009). Security Risk Management Body of Knowledge. *John Wiley & Sons*.
- Thadt. (2020, February 9). Software Analysis (Basic Introduction). *Medium*. URL: <https://medium.com/@thadt/software-analysis-basic-introduction-dea3a735503a> (Retrieved: 21.2.2024)
- Tufano, M.; Agarwal, A.; Jang, J.; Moghaddam, R. Z.; & Sundaresan, N. (2024, March 13). AutoDev: Automated AI-Driven Development. *arXiv, Computer Science, Software Engineering*. DOI: <https://doi.org/10.48550/arXiv.2403.08299>
- Vogelsang, A; & Borg, M. (2019). Requirements Engineering for Machine Learning: Perspectives from Data Scientists. *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. DOI: <http://doi.org/10.1109/REW.2019.00050>
- Wan, Z.; Xia, X.; Lo D.; & Murphy, G. C. (2021, September 1). How does Machine Learning Change Software Development Practices? *IEEE Transactions on Software Engineering, Volume 47, Issue 9, Pages 1857-1871*. DOI: <https://doi.org/10.1109/TSE.2019.2937083>
- Wiklund, K.; Eldh, S.; Sundmark, D.; & Lundqvist, K. (2017, December). Impediments for software test automation: A systematic literature review. *Software: Testing, Verification and Reliability, Volume 27, Issue 8*. DOI: <https://doi.org/10.1002/stvr.1639>
- Yahav, E. (2015). Programming with “Big Code”. In: Feng, X., Park, S. (eds) *Programming Languages and Systems. APLAS 2015. Lecture Notes in Computer Science()*, Volume 9458. Springer, Cham. DOI: https://doi.org/10.1007/978-3-319-26529-2_1
- Zhang, D. (2000, June 16th). Applying machine learning algorithms in software development. *Proceedings of the 2000 Monterey Workshop on Modelling Software System Structures in a fastly moving scenario, Santa Margherita Ligure, Italy*.

Zhin, J. (2018). Requirements Engineering Methodologies. In Environment Modeling-Based Requirements Engineering for Software Intensive Systems (Pages 13-27). Elsevier Inc. URL: <https://www.sciencedirect.com/topics/computer-science/requirement-engineering>